Estudos Fundamentais do DIDGuard

Apostila técnica por áreas de estudo

Alysson Machado

2026

Apresentação

Esta apostila organiza os fundamentos técnicos necessários para compreender e construir soluções alinhadas ao DIDGuard. Cada capítulo cobre conceitos, terminologia, referências e modelos de implementação, com foco em estudo dirigido e base conceitual sólida.

O percurso foi desenhado para que cada capítulo prepare o terreno para o seguinte:

Cap. 1 apresenta **Identificadores Descentralizados (DIDs)** — o alicerce de identidade do sistema;

Cap. 2 explora **Tags RFID e MIFARE Classic** — a camada física que transporta credenciais;

Cap. 3 introduz **Criptografia Aplicada** — as primitivas que protegem dados e identidades;

Cap. 4 aborda **Blockchain e Contratos Inteligentes** — o registro verificável de permissões;

Cap. 5 apresenta **ESP32, ESP-IDF e PlatformIO** — a base de firmware da borda conectada.

Todos os capítulos incluem uma seção "Rodando na prática" com laboratórios autocontidos no diretório `examples/`, permitindo que o leitor experimente cada conceito antes de avançar.

Sumário

Lista de Siglas **viii**

1 DIDs: fundamentos, estrutura e resolução **1**

1.1 Objetivos de aprendizagem 1
1.2 O que é um DID 1
1.3 Por que usar DIDs 2
1.4 DID Document 2
1.5 Exemplo completo de DID Document 3
1.6 Explicação dos campos do exemplo 4
1.7 Campos mínimos e extensões 5
1.8 Resolução de DID 5
1.9 Saídas da resolução 6
1.10 Fluxo genérico de resolução 6
1.11 Exemplos de resolução por método 6
1.12 Resolução em arquiteturas com registro verificável e IPFS 7
1.13 Resolução versus dereferenciamento 7
1.14 Cuidados técnicos no processo de resolução 8
1.15 Boas práticas de modelagem 8
1.16 Exercícios sugeridos 8
1.17 Leituras recomendadas 8
1.18 Rodando na prática 9
1.18.1 Pré-requisitos 9
1.18.2 Arquivos do laboratório 9
1.18.3 Executando 9
1.18.4 O que observar 10
1.18.5 Perguntas de análise 10
1.18.6 Extensão sugerida 10
1.18.7 Localização do exemplo 10

2 Tags RFID e MIFARE Classic **12**

2.1 Objetivos de aprendizagem 12

2.2 Visão geral 12
2.3 Componentes do hardware 13
2.4 Fluxo básico de comunicação 13
2.5 Organização da memória na MIFARE Classic 14
2.5.1 MIFARE Classic 1K 14
2.5.2 MIFARE Classic 4K 15
2.6 Setor trailer e bits de acesso 15
2.7 Blocos de valor 16
2.8 Comandos básicos no nível da tag 16
2.9 Registradores relevantes do MFRC522 16
2.10 Exemplo de mapeamento de aplicação 17
2.11 Limitações de segurança da MIFARE Classic 18
2.12 Por que reforçar a criptografia no nível da aplicação 18
2.13 MIFARE Classic versus plataformas mais modernas 19
2.14 Leituras recomendadas 19
2.15 Rodando na prática 20
2.15.1 Pré-requisitos 20
2.15.2 Arquivos do laboratório 20
2.15.3 Executando 20
2.15.4 O que observar 20
2.15.5 Perguntas de análise 21
2.15.6 Extensão sugerida 21
2.15.7 Conexão com a arquitetura segura 21
2.15.8 Localização do exemplo 21

3 Fundamentos de Criptografia Aplicada 22
3.1 Objetivos de aprendizagem 22
3.2 Propriedades de segurança fundamentais 22
3.3 Funções hash criptográficas 23
3.3.1 Propriedades esperadas 23
3.3.2 SHA-256 23
3.4 Hash não é criptografia de sigilo 24
3.5 Código de autenticação de mensagem (MAC) e HMAC 24
3.5.1 Intuição de uso 24
3.5.2 Quando HMAC é útil 25
3.6 Criptografia simétrica 25
3.6.1 AES-GCM 25
3.7 Criptografia assimétrica 26
3.7.1 O que a assimetria permite 26

3.8 Curvas elípticas e ECDSA . . . 26
3.8.1 Por que curvas elípticas são populares . . . 26
3.8.2 Assinatura digital em alto nível . . . 27
3.9 Nonce, contador e anti-replay . . . 27
3.9.1 Exemplo conceitual . . . 27
3.9.2 Requisitos de projeto . . . 27
3.10 Derivação e gestão de chaves . . . 28
3.11 Aplicação desses conceitos em uma arquitetura como o DIDGuard . . . 28
3.12 Erros comuns em projetos iniciantes . . . 29
3.13 Leituras recomendadas . . . 29
3.14 Rodando na prática . . . 30
3.14.1 Pré-requisitos . . . 30
3.14.2 Arquivos do laboratório . . . 30
3.14.3 Executando . . . 30
3.14.4 O que observar . . . 30
3.14.5 Perguntas de análise . . . 30
3.14.6 Extensão sugerida . . . 31
3.14.7 Localização do exemplo . . . 31

4 Blockchain e Contratos Inteligentes 32
4.1 Objetivos de aprendizagem . . . 32
4.2 Do livro-razão distribuído ao contrato inteligente . . . 32
4.3 Elementos básicos de uma blockchain . . . 33
4.4 Por que blockchain não é apenas banco de dados . . . 33
4.5 Transações, contas e estado . . . 34
4.6 Consenso em alto nível . . . 34
4.7 Ethereum e a Ethereum Virtual Machine . . . 34
4.8 Gas e custo computacional . . . 35
4.8.1 Por que gas existe . . . 35
4.8.2 Impacto no desenho de contratos . . . 35
4.9 Contratos inteligentes . . . 35
4.10 Leitura, escrita e eventos . . . 36
4.10.1 Funções de leitura . . . 36
4.10.2 Funções de escrita . . . 36
4.10.3 Eventos . . . 37
4.11 Estruturas típicas em contratos . . . 37
4.12 Armazenamento on-chain versus off-chain . . . 37
4.13 Por que blockchain faz sentido em controle de acesso . . . 38
4.14 Limitações e trade-offs . . . 38

4.15 Relação com uma arquitetura como a do DIDGuard . . . 39
4.16 Leituras recomendadas . . . 39
4.17 Rodando na prática . . . 40
4.17.1 Pré-requisitos . . . 40
4.17.2 Arquivos do laboratório . . . 40
4.17.3 Instalação . . . 40
4.17.4 Execução . . . 40
4.17.5 O que observar . . . 41
4.17.6 Perguntas de análise . . . 41
4.17.7 Extensão sugerida . . . 41
4.17.8 Localização do exemplo . . . 41

5 ESP32, ESP-IDF e PlatformIO 43
5.1 Objetivos de aprendizagem . . . 43
5.2 Por que o firmware merece um capítulo próprio . . . 43
5.3 Visão geral do ESP32 . . . 44
5.4 ESP-IDF: o framework oficial . . . 44
5.5 PlatformIO como camada de produtividade . . . 44
5.6 Estrutura típica de projeto . . . 45
5.7 Configuração via `platformio.ini` . . . 45
5.8 Configuração persistente via `sdkconfig.defaults` . . . 46
5.9 Inicialização básica do sistema . . . 46
5.10 Exemplo: conexão Wi-Fi em modo estação . . . 46
5.11 Integração com o RC522 via SPI . . . 47
5.11.1 Por que usar a biblioteca como componente . . . 49
5.11.2 Acesso direto versus componente . . . 49
5.12 Criptografia local no ESP32 . . . 49
5.12.1 Geração de chave elíptica . . . 50
5.12.2 Derivação de um identificador . . . 50
5.13 Montando um DID Document no firmware . . . 50
5.14 Fluxo conceitual completo . . . 51
5.15 Boas práticas de projeto . . . 51
5.16 Leituras recomendadas . . . 52
5.17 Rodando na prática . . . 52
5.17.1 Pré-requisitos . . . 52
5.17.2 Arquivos do laboratório . . . 53
5.17.3 O que o aluno deve observar . . . 53
5.17.4 Execução sugerida . . . 53
5.17.5 Resultados esperados . . . 53

5.17.6 Extensão sugerida 54
5.17.7 Localização do exemplo 54

Glossário do Capítulo 1 **55**

Glossário do Capítulo 2 **58**

Glossário do Capítulo 3 **61**

Glossário do Capítulo 4 **65**

Glossário do Capítulo 5 **68**

Lista de Siglas

ABI	*Application Binary Interface*
AES	*Advanced Encryption Standard*
API	Interface de programacao de aplicacoes
CID	*Content Identifier*
CRC	*Cyclic Redundancy Check*
DHT	*Distributed Hash Table*
DID	*Decentralized Identifier*
EC	*Elliptic Curve*
ECDSA	*Elliptic Curve Digital Signature Algorithm*
EEPROM	*Electrically Erasable Programmable Read-Only Memory*
ESP-IDF	*Espressif IoT Development Framework*
ESP32	Plataforma de microcontrolador com Wi-Fi da Espressif
EVM	*Ethereum Virtual Machine*
FIFO	*First In, First Out*
FIPS	*Federal Information Processing Standards*
GPIO	*General-Purpose Input/Output*
GCM	*Galois/Counter Mode*
GMAC	*Galois Message Authentication Code*

HF	Alta frequencia
HMAC	*Hash-based Message Authentication Code*
HTTP	*Hypertext Transfer Protocol*
HTTPS	*Hypertext Transfer Protocol Secure*
IPFS	*InterPlanetary File System*
IRQ	*Interrupt Request*
JSON	*JavaScript Object Notation*
JWK	*JSON Web Key*
JWS	*JSON Web Signature*
MAC	Codigo de autenticacao de mensagem
MFRC522	Leitor RFID MFRC522 da NXP
MIFARE	Familia de cartoes e tags sem contato da NXP
MISO	*Master in, slave out*
MOSI	*Master out, slave in*
NIST	*National Institute of Standards and Technology*
NSS	*Slave select*
NUID	*Non-Unique Identifier*
NVS	*Non-Volatile Storage*
PCD	*Proximity Coupling Device*
PICC	*Proximity Integrated Circuit Card*
RFC	*Request for Comments*
RFID	*Radio Frequency Identification*
RST	*Reset*

RTOS	*Real-Time Operating System*
SCK	*Serial clock*
SDK	*Software Development Kit*
SDA	Rotulo de selecao do periferico em modulos RC522
SHA	*Secure Hash Algorithm*
SoC	*System on Chip*
SPI	*Serial Peripheral Interface*
STA	*Station mode*
UID	*Unique Identifier*
URI	*Uniform Resource Identifier*
W3C	*World Wide Web Consortium*

Capítulo 1

DIDs: fundamentos, estrutura e resolução

1.1 Objetivos de aprendizagem

Ao final deste capítulo, o aluno deve ser capaz de:

- explicar o que é um *Decentralized Identifier* (DID) e por que ele difere de identificadores centralizados;
- descrever os elementos que compõem um DID Document;
- interpretar um exemplo completo de DID Document conforme o padrão;
- explicar o processo de resolução de DID e seus principais atores;
- distinguir resolução de DID, dereferenciamento e acesso a serviços vinculados.

1.2 O que é um DID

Um DID (*Decentralized Identifier*) é um identificador global, controlado criptograficamente, que não depende de uma autoridade central única para sua gestão. De acordo com o padrão DID Core, publicado pelo *World Wide Web Consortium* (W3C), um DID identifica um sujeito — pessoa, organização, dispositivo, agente de software ou outro recurso — e aponta para um DID Document com metadados necessários para verificação e interação [World Wide Web Consortium (W3C), 2022].

A forma geral de um DID é:

```
did:<método>:<identificador-específico>
```

Exemplos:

- `did:web:example.com`
- `did:key:z6Mk...`
- `did:example:123456789abcdefghi`

Cada método DID define como o identificador é criado, resolvido, atualizado e, quando aplicável, desativado. Por isso, a semântica operacional de um DID sempre depende do método associado.

1.3 Por que usar DIDs

Identificadores tradicionais costumam depender de diretórios centralizados, provedores de identidade ou bancos de dados administrados por uma única organização. DIDs foram propostos para viabilizar identidades mais portáveis, verificáveis e interoperáveis.

Entre as vantagens arquiteturais mais importantes, destacam-se:

- **Controle criptográfico**: a autenticação e a prova de controle dependem de chaves e assinaturas, e não apenas de cadastro em base central;
- **Portabilidade**: o identificador pode ser reutilizado entre sistemas diferentes, desde que compartilhem o método e as regras de resolução;
- **Verificabilidade**: chaves, serviços e vínculos podem ser auditados com base em documentos e evidências técnicas;
- **Separação entre identidade e aplicação**: o DID representa a identidade; as regras de negócio ficam em camadas superiores.

1.4 DID Document

O DID Document é a representação estruturada dos metadados associados a um DID. Em implementações práticas, ele costuma ser serializado em *JavaScript Object Notation* (JSON). Ele informa como um sujeito pode ser autenticado, quais métodos de verificação são válidos e quais serviços podem ser descobertos por meio daquela identidade.

Nem todo DID Document terá exatamente os mesmos campos. O conteúdo depende:

- do método DID;
- do domínio da aplicação;
- dos mecanismos de autenticação e prova adotados;
- das extensões e registries utilizados.

1.5 Exemplo completo de DID Document

O exemplo a seguir ilustra um DID Document didático contendo os campos mais comuns do padrão:

Antes de examinar o JSON, vale notar três siglas presentes no exemplo: *JSON Web Key* (JWK), usado no campo `publicKeyJwk`; *JSON Web Signature* (JWS), referenciado no contexto `jws-2020`; e *Elliptic Curve* (EC), indicado no campo `kty`.

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/suites/jws-2020/v1"
  ],
  "id": "did:example:123456789abcdefghi",
  "controller": "did:example:123456789abcdefghi",
  "verificationMethod": [
    {
      "id": "did:example:123456789abcdefghi#key-1",
      "type": "JsonWebKey2020",
      "controller": "did:example:123456789abcdefghi",
      "publicKeyJwk": {
        "kty": "EC",
        "crv": "P-256",
        "x": "f83OJ3D2xF4r5n6Q7s8t9u0v1w2y3z4A5B6C7D8E9F0",
        "y": "x_FEzRu9h2QvW0Ykq1c2d3e4f5g6h7i8j9kLmNoPqRs"
      }
    }
  ],
  "authentication": [
    "did:example:123456789abcdefghi#key-1"
  ],
  "assertionMethod": [
    "did:example:123456789abcdefghi#key-1"
  ],
  "keyAgreement": [
    "did:example:123456789abcdefghi#key-1"
  ],
  "service": [
    {
      "id": "did:example:123456789abcdefghi#messaging",
```

```
      "type": "MessagingService",
      "serviceEndpoint": "https://example.org/messages"
    }
  ]
}
```

1.6 Explicação dos campos do exemplo

- `@context`: define os contextos semânticos usados na interpretação do documento. O primeiro contexto normalmente aponta para o núcleo do padrão DID. No exemplo, o segundo contexto referencia a suíte *JSON Web Signature* (JWS) 2020.
- `id`: é o próprio DID que este documento descreve. Deve coincidir com o identificador resolvido.
- `controller`: informa quem controla o DID ou o recurso descrito. Em muitos casos, o controlador coincide com o próprio `id`, mas isso não é obrigatório.
- `verificationMethod`: lista os métodos de verificação associados ao DID. Cada entrada descreve uma chave ou mecanismo de verificação.
- `verificationMethod[].id`: identificador único do método de verificação. Frequentemente usa um fragmento, como `#key-1`.
- `verificationMethod[].type`: tipo do método de verificação, conforme registries e suites criptográficas aceitas.
- `verificationMethod[].controller`: entidade controladora daquele método específico.
- `publicKeyJwk`: representação da chave pública em formato *JSON Web Key* (JWK). Em outros documentos, a chave pode aparecer em formatos alternativos, como `publicKeyMultibase`. No exemplo, o valor `EC` em `kty` indica o uso de *Elliptic Curve*.
- `authentication`: lista quais métodos podem ser usados para autenticação do sujeito.
- `assertionMethod`: lista os métodos autorizados para emitir ou assinar afirmações, como credenciais e declarações verificáveis.
- `keyAgreement`: lista métodos aptos a estabelecer segredos compartilhados, em cenários de troca segura de mensagens.
- `service`: lista pontos de serviço vinculados ao DID, como endpoints de mensageria, descoberta, resolução auxiliar ou interfaces de programação de aplicações (APIs) específicas.

- `serviceEndpoint`: endereço do serviço associado. Não é, por si só, prova de autenticidade; é apenas um vínculo declarado pelo documento.

1.7 Campos mínimos e extensões

Na prática, um DID Document pode ser mais enxuto ou mais rico do que o exemplo anterior. Há documentos que incluem apenas:

- `@context`;
- `id`;
- um ou mais `verificationMethod`;
- uma forma de autenticação.

Por outro lado, aplicações específicas podem acrescentar:

- serviços personalizados;
- chaves segregadas por propósito;
- metadados operacionais;
- vínculos com infraestrutura de armazenamento distribuído ou registradores externos.

1.8 Resolução de DID

A resolução de DID é o processo de obter, a partir de um DID, o DID Document correspondente e os metadados associados a essa operação. Em termos conceituais, resolver um DID significa responder à pergunta: *"qual é o documento atual e verificável que descreve esta identidade?"*

Esse processo envolve, em geral, quatro atores:

- **Cliente**: aplicação que precisa do DID Document;
- **Resolver**: componente que interpreta o método DID e coordena a busca;
- **Driver ou adaptador do método**: implementação específica de cada método;
- **Fonte de verdade**: infraestrutura onde o estado do DID está registrado, como *Hypertext Transfer Protocol* (HTTP), blockchain, *Distributed Hash Table* (DHT), banco distribuído ou outro registro verificável.

1.9 Saídas da resolução

Em uma implementação completa, a resolução não devolve apenas o DID Document. Ela costuma produzir três grupos de informação:

- **DID Document**: o conteúdo resolvido;
- **Metadados do documento**: informações como versão, atualização, desativação ou equivalências;
- **Metadados da resolução**: informações sobre o próprio processo, incluindo erros, método usado, cache e opções de resolução.

1.10 Fluxo genérico de resolução

Embora cada método DID tenha seu próprio comportamento, o fluxo genérico costuma seguir a sequência abaixo:

1. o cliente fornece um DID ao resolvedor;
2. o resolvedor identifica o método, por exemplo `web`, `key` ou outro;
3. o adaptador daquele método aplica as regras de resolução correspondentes;
4. o sistema consulta a fonte de verdade do método;
5. o documento obtido é validado e normalizado;
6. o resultado é devolvido ao cliente com documento e metadados.

1.11 Exemplos de resolução por método

- **`did:web`**: a resolução é feita via *Hypertext Transfer Protocol Secure* (HTTPS), normalmente recuperando um arquivo como `/.well-known/did.json` ou caminho equivalente no domínio.
- **`did:key`**: a resolução é local e determinística, pois a chave pública já está embutida no próprio identificador.
- **Métodos ancorados em blockchain**: a resolução exige consulta ao registro on-chain, leitura de eventos, estados ou apontadores para dados externos.

1.12 Resolução em arquiteturas com registro verificável e IPFS

Em arquiteturas didáticas ou experimentais, é comum separar:

- um **registro verificável**, responsável por manter o vínculo entre DID e uma referência estável;
- um **armazenamento distribuído**, como o *InterPlanetary File System* (IPFS), responsável por hospedar o documento completo.

Um fluxo típico é:

1. resolver o DID no registro verificável;
2. obter um ponteiro, como um hash, um *Content Identifier* (CID) ou um identificador de versão;
3. recuperar o documento na camada de armazenamento;
4. validar se o documento recuperado é coerente com o vínculo registrado.

Essa abordagem ajuda a separar governança de identidade, auditabilidade e armazenamento por conteúdo.

1.13 Resolução versus dereferenciamento

É importante não confundir resolução com dereferenciamento.

- **Resolução**: retorna o DID Document associado a um DID.
- **Dereferenciamento**: resolve um recurso específico dentro desse universo, como um fragmento `#key-1` ou um serviço particular.

Exemplo:

- resolver `did:example:123456789abcdefghi` significa obter o DID Document completo;
- dereferenciar `did:example:123456789abcdefghi#key-1` significa obter o método de verificação específico apontado pelo fragmento.

1.14 Cuidados técnicos no processo de resolução

Ao projetar ou implementar resolução de DID, alguns cuidados são essenciais:

- validar a autenticidade da fonte de verdade;
- tratar indisponibilidade, timeout e inconsistência entre registro e documento;
- registrar metadados de erro e rastreabilidade da resolução;
- considerar cache com critério, para não servir documentos desatualizados;
- proteger informações sensíveis que não devam ser publicadas em documentos resolvíveis.

1.15 Boas práticas de modelagem

- manter o DID Document focado em dados necessários para verificação e descoberta;
- separar dados sensíveis de metadados públicos;
- escolher formatos de chave e serviços alinhados ao ecossistema do método DID;
- documentar critérios de atualização, revogação e desativação.

1.16 Exercícios sugeridos

1. comparar os métodos `did:key`, `did:web` e um método ancorado em blockchain;
2. identificar, no exemplo deste capítulo, quais campos são voltados para autenticação, quais são voltados para assinatura de afirmações e quais são voltados para descoberta de serviços;
3. desenhar um fluxo de resolução para uma arquitetura que use contrato inteligente e IPFS;
4. explicar, com um diagrama simples, a diferença entre resolução e dereferenciamento.

1.17 Leituras recomendadas

- W3C DID Core, norma principal sobre modelo de dados e semântica [World Wide Web Consortium (W3C), 2022];
- W3C DID Specification Registries, para métodos, tipos e registries de apoio [World Wide Web Consortium (W3C), 2024];

- DID Resolution, para aprofundamento do processo de resolução e metadados [World Wide Web Consortium (W3C), 2026];
- *Request for Comments* (RFC) 3986, para sintaxe geral de identificadores *Uniform Resource Identifier* (URI) [Berners-Lee et al., 2005];
- W3C Verifiable Credentials Data Model, para integração entre identidade e credenciais verificáveis [World Wide Web Consortium (W3C), 2023].

1.18 Rodando na prática

O objetivo desta prática é executar um resolvedor mínimo de `did:web`, publicar localmente um *DID Document* e observar, passo a passo, como um DID é convertido em um recurso resolvível.

1.18.1 Pré-requisitos

- Node.js 18 ou superior;
- terminal com suporte aos comandos do Node;
- diretório `docs/book/examples/cap1-did-resolution` disponível localmente.

1.18.2 Arquivos do laboratório

O mini projeto deste capítulo contém:

```
did.json
server.mjs
resolve.mjs
```

1.18.3 Executando

No diretório do laboratório, abra dois terminais. No primeiro:

```
node server.mjs
```

No segundo:

```
node resolve.mjs did:web:localhost%3A8081:users:alice
```

1.18.4 O que observar

- a regra de transformação do identificador `did:web` em URL;
- a recuperação do documento `did.json` por HTTP;
- a estrutura retornada, com `id`, `verificationMethod`, `authentication` e `service`;
- a diferença entre o identificador abstrato e o recurso concreto acessado durante a resolução.

1.18.5 Perguntas de análise

- Por que o laboratório usa HTTP em vez de HTTPS?
- Qual é a diferença entre resolver um DID e dereferenciar um fragmento como `#key-1`?
- O que aconteceria se o arquivo `did.json` não contivesse o campo `id`?
- Como esse mecanismo de resolução se compara a uma consulta em diretório centralizado?

1.18.6 Extensão sugerida

Altere o arquivo `did.json` para:

- adicionar um novo `serviceEndpoint`;
- trocar o identificador do controlador;
- incluir uma segunda chave em `verificationMethod`.

Depois repita a execução e verifique como a mudança afeta a saída do resolvedor.

1.18.7 Localização do exemplo

Todos os arquivos usados nesta prática estão em:

`docs/book/examples/cap1-did-resolution`

Transição para o próximo capítulo. Até aqui, o foco esteve na camada lógica da identidade: como um DID é estruturado, publicado e resolvido. No entanto, em um sistema de controle de acesso físico como o DIDGuard, essa identidade precisa estar associada a uma credencial tangível — um cartão, tag ou dispositivo que o portador apresenta a um

leitor. O próximo capítulo explora essa camada física, investigando como as tags RFID MIFARE Classic organizam sua memória, autenticam setores e — sobretudo — por que suas proteções nativas não são suficientes quando usadas isoladamente.

Capítulo 2

Tags RFID e MIFARE Classic

2.1 Objetivos de aprendizagem

Ao final deste capítulo, o aluno deve ser capaz de:

- explicar o funcionamento básico de sistemas de *Radio Frequency Identification* (RFID) em 13,56 MHz;
- distinguir os papéis do leitor e da tag em uma arquitetura com MIFARE Classic;
- interpretar a organização em setores, blocos e setor trailer de uma MIFARE Classic;
- identificar registradores essenciais do leitor MFRC522 e sua função no fluxo de leitura e escrita;
- justificar por que uma aplicação moderna costuma adicionar uma camada criptográfica própria sobre a tag.

2.2 Visão geral

RFID é uma tecnologia de identificação por radiofrequência que permite trocar dados entre um leitor e uma credencial sem contato físico direto. No contexto deste capítulo, o foco está na faixa de alta frequência (HF), operando em 13,56 MHz, bastante comum em cartões de acesso, bilhetagem, laboratórios e kits educacionais [Finkenzeller, 2010, NXP Semiconductors N.V., 2018].

Dentro desse ecossistema, a família MIFARE Classic é uma das mais conhecidas em ambientes didáticos. Ela combina baixo custo, grande disponibilidade de módulos e uma organização de memória simples o suficiente para estudo de autenticação, leitura, escrita e controle de acesso. Ao mesmo tempo, suas limitações de segurança são bem documentadas na literatura, o que a torna um bom objeto de estudo para discutir por que

sistemas reais precisam de reforços adicionais na camada de aplicação [Garcia et al., 2008, de Koning Gans et al., 2008].

2.3 Componentes do hardware

Em uma arquitetura típica com MIFARE Classic, há pelo menos quatro elementos:

- **Leitor**: o *Proximity Coupling Device* (PCD), responsável por energizar a tag, iniciar a comunicação e executar comandos de leitura, escrita e autenticação.
- **Tag ou cartão**: o *Proximity Integrated Circuit Card* (PICC), que responde ao leitor usando o campo eletromagnético recebido.
- **Controlador hospedeiro**: um microcontrolador ou computador que configura o leitor, interpreta respostas e integra o fluxo com a aplicação.
- **Antena e estágio de radiofrequência**: circuito que gera o campo em 13,56 MHz e sustenta o acoplamento entre leitor e tag.

Em laboratório, é comum usar o circuito leitor MFRC522, controlado por um microcontrolador via barramento *Serial Peripheral Interface* (SPI). Em módulos didáticos, os sinais mais frequentes são:

- `SCK` (*serial clock*), `MOSI` (*master out, slave in*) e `MISO` (*master in, slave out*) para o barramento SPI;
- `SDA`, rótulo adotado por alguns módulos para a linha de seleção do periférico, ou `NSS` (*slave select*);
- `RST` (*reset*) para reinicialização do leitor;
- linha de *Interrupt Request* (IRQ) opcional para sinalização de eventos assíncronos.

O MFRC522 é o circuito de front-end que cuida da parte de radiofrequência, temporização, fila interna de dados e comandos do protocolo ISO/IEC 14443 A, enquanto o microcontrolador hospedeiro fica responsável pela lógica da aplicação [NXP Semiconductors N.V., 2016].

2.4 Fluxo básico de comunicação

O fluxo mínimo entre leitor e tag pode ser descrito em etapas:

1. o leitor ativa o campo de radiofrequência e entra em estado de busca;

2. a tag no campo responde a uma requisição inicial;
3. o leitor executa o procedimento de anticolisão para descobrir e selecionar uma tag específica;
4. a partir do *Unique Identifier* (UID) da tag selecionada, o sistema decide o que fazer em seguida;
5. para acessar setores protegidos, o leitor executa autenticação com `Key A` ou `Key B`;
6. somente depois da autenticação o leitor pode ler, escrever ou executar operações especiais sobre os blocos permitidos.

Esse ponto é importante: o UID ajuda a selecionar a tag, mas não deve ser tratado, sozinho, como prova suficiente de identidade ou autorização. Em especial, aplicações de controle de acesso que confiam apenas no UID tendem a ser frágeis.

2.5 Organização da memória na MIFARE Classic

As tags MIFARE Classic armazenam dados em memória não volátil do tipo *Electrically Erasable Programmable Read-Only Memory* (EEPROM). A unidade básica é o bloco de 16 bytes. Esses blocos são agrupados em setores [NXP Semiconductors N.V., 2018].

2.5.1 MIFARE Classic 1K

Na variante 1K, com aproximadamente 1 kilobyte de capacidade nominal, a organização mais comum é:

- 16 setores;
- 4 blocos por setor;
- 16 bytes por bloco;
- total nominal de 1024 bytes.

O arranjo de um setor típico é:

```
Bloco 0   Dados ou, no setor 0, bloco de fabricante
Bloco 1   Dados
Bloco 2   Dados
Bloco 3   Setor trailer
```

No setor 0, bloco 0, fica o bloco de fabricante. Ele contém informações gravadas de fábrica, incluindo o UID ou um identificador compatível, e não deve ser tratado como área comum de dados da aplicação.

2.5.2 MIFARE Classic 4K

Na variante 4K, com aproximadamente 4 kilobytes de capacidade nominal, a organização é híbrida:

- setores 0 a 31 com 4 blocos por setor;
- setores 32 a 39 com 16 blocos por setor;
- 16 bytes por bloco em toda a memória.

Essa diferença é relevante porque a lógica de endereçamento, autenticação e mapeamento de blocos precisa considerar o tamanho do setor.

2.6 Setor trailer e bits de acesso

O último bloco de cada setor é chamado de setor trailer. Ele concentra o material de controle de acesso do setor e costuma ser a área mais sensível do ponto de vista de configuração.

Sua estrutura típica é:

```
Bytes 0..5    Key A
Bytes 6..8    Bits de acesso e seus complementos
Byte  9       General Purpose Byte
Bytes 10..15  Key B
```

Os bits de acesso definem quem pode:

- ler ou escrever blocos de dados;
- usar `Key A` ou `Key B` para autenticação;
- ler ou modificar partes do próprio trailer;
- executar operações especiais, como atualização de blocos de valor.

Há duas observações práticas importantes:

- a codificação dos bits de acesso usa redundância e complementos, então erros de configuração podem tornar o setor inacessível;
- a escolha de `Key A` e `Key B` não substitui uma política de aplicação: ela controla o acesso ao setor, mas não representa, sozinha, uma identidade forte de usuário.

2.7 Blocos de valor

Além de blocos comuns de dados, a MIFARE Classic admite o formato de bloco de valor, pensado para armazenar contadores, créditos e débitos. Esse formato usa redundância interna para detectar erros simples de integridade:

```
Bytes 0..3    Valor
Bytes 4..7    Complemento do valor
Bytes 8..11   Repetição do valor
Bytes 12..15  Endereço do bloco e seus complementos
```

Esse desenho permite operações como incremento, decremento, restauração e transferência. Ainda assim, ele não resolve problemas maiores de autenticação forte, rastreabilidade ou vínculo com regras de negócio externas.

2.8 Comandos básicos no nível da tag

Sem entrar em detalhes operacionais de baixo nível, o conjunto mínimo de operações estudado em laboratório costuma incluir:

- requisição e ativação da tag;
- anticolisão e seleção;
- autenticação com `Key A` ou `Key B`;
- leitura de bloco;
- escrita de bloco;
- operações sobre bloco de valor.

Do ponto de vista do projeto de sistemas, essas operações bastam para mostrar que a tag oferece armazenamento e controle básico de acesso por setor, mas não entrega, por si só, semântica rica de autorização.

2.9 Registradores relevantes do MFRC522

No leitor MFRC522, o microcontrolador hospedeiro controla o fluxo por meio de registradores. Eles pertencem ao leitor, não à tag. Em outras palavras, não devem ser confundidos com os blocos de memória da MIFARE Classic [NXP Semiconductors N.V., 2016].

Registrador	Endereço	Papel básico
`CommandReg`	`0x01`	Seleciona o comando interno do leitor, como ocioso, transmissão, recepção, autenticação ou cálculo de CRC.
`ComIrqReg`	`0x04`	Indica eventos concluídos, temporização, recepção ou transmissão e ajuda a sincronizar o firmware com o estado do leitor.
`ErrorReg`	`0x06`	Reporta condições de erro, como colisão, paridade, buffer ou protocolo.
`Status1Reg`	`0x07`	Expõe estados internos úteis para acompanhar a máquina de estados do leitor.
`FIFODataReg`	`0x09`	Guarda os bytes enviados ao leitor e os bytes recebidos da tag na fila *First In, First Out* (FIFO).
`FIFOLevelReg`	`0x0A`	Informa quantos bytes estão presentes na FIFO e permite zerar a fila quando necessário.
`BitFramingReg`	`0x0D`	Controla alinhamento de bits, número de bits válidos do último byte e início de transmissão.
`ModeReg`	`0x11`	Configura aspectos globais do modo de operação, incluindo comportamento do cálculo de *Cyclic Redundancy Check* (CRC).
`TxModeReg` e `RxModeReg`	`0x12` e `0x13`	Ajustam parâmetros de transmissão e recepção do protocolo.
`TxControlReg`	`0x14`	Liga ou desliga a antena e controla o estágio transmissor.
`CRCResultRegH/L`	`0x21` e `0x22`	Disponibilizam o resultado do cálculo de CRC realizado pelo próprio leitor.
`VersionReg`	`0x37`	Informa a versão do circuito, útil em diagnóstico e compatibilidade.

Na prática, uma biblioteca de driver esconde boa parte desse detalhe. Mesmo assim, conhecer esses registradores ajuda o aluno a entender como o firmware controla a sequência de autenticação, leitura, escrita e tratamento de falhas.

2.10 Exemplo de mapeamento de aplicação

Uma aplicação pode reservar blocos específicos para seus próprios dados. Um desenho didático possível é:

```
Bloco 1   Identificador da aplicação
Bloco 2   Timestamp ou versão
Bloco 4   Código de autenticação de mensagem (MAC) ou resumo de integridade
```

```
Bloco 13  Material de verificação adicional
Bloco 14  Estado de segurança, como contador e MAC do contador
Blocos 16+ Assinaturas ou chaves públicas derivadas
```

Esse tipo de mapeamento já mostra uma ideia importante: a tag deixa de ser apenas um repositório de bytes arbitrários e passa a participar de um protocolo maior, desenhado pela aplicação.

2.11 Limitações de segurança da MIFARE Classic

Do ponto de vista histórico, a MIFARE Classic teve grande adoção prática. Do ponto de vista de segurança moderna, porém, ela não deve ser tratada como única raiz de confiança. Entre as limitações mais relevantes estão:

- uso da cifra proprietária CRYPTO1, hoje amplamente estudada e considerada inadequada para novas arquiteturas que exijam alto nível de proteção;
- dependência frequente de chaves estáticas por setor;
- tendência, em sistemas simplificados, de confiar apenas no UID como identificador de usuário;
- dificuldade de expressar políticas ricas, como validade temporal, vínculo com dispositivo, auditoria e múltiplos fatores.

A literatura técnica mostrou, há bastante tempo, que o modelo de segurança da MIFARE Classic possui fragilidades estruturais. Em consequência, seu uso como credencial única de acesso deve ser evitado em sistemas que demandem maior resistência a clonagem, replay ou engenharia reversa [Garcia et al., 2008, de Koning Gans et al., 2008].

2.12 Por que reforçar a criptografia no nível da aplicação

É exatamente nesse ponto que entra o gancho arquitetural de sistemas como o DIDGuard. Em vez de confiar que a segurança nativa da MIFARE Classic basta, a aplicação adiciona uma camada própria de proteção e validação.

Essa camada pode oferecer:

- associação da tag a uma identidade lógica mais rica do que o UID;
- mensagens autenticadas com *Hash-based Message Authentication Code* (HMAC) ou assinaturas digitais;

- contadores monotônicos e desafios aleatórios para reduzir replay;
- verificação de autorização fora da tag, em um serviço, contrato ou política central verificável;
- auditoria, revogação e validade temporal das permissões;
- segundo fator, como biometria, em fluxos de risco maior.

Em termos práticos, a MIFARE Classic pode continuar sendo usada como portadora de um identificador e de material auxiliar de autenticação, mas a decisão de acesso passa a depender de um protocolo mais forte no nível da aplicação. Isso evita superestimar a proteção oferecida por uma tecnologia originalmente pensada para cenários mais simples.

2.13 MIFARE Classic versus plataformas mais modernas

Cartões mais recentes da própria linha MIFARE, como variantes baseadas em *Advanced Encryption Standard* (AES), oferecem mecanismos criptográficos e operacionais mais robustos do que a MIFARE Classic [NXP Semiconductors, 2021]. Ainda assim, mesmo com uma tag mais forte, continua fazendo sentido manter uma camada de aplicação bem desenhada para:

- vincular credenciais a regras de negócio;
- separar autenticação da decisão de autorização;
- integrar rastreabilidade, revogação e múltiplos fatores;
- reduzir o impacto de comprometimento de um único componente físico.

2.14 Leituras recomendadas

- Finkenzeller, para fundamentos gerais de RFID, arquitetura e protocolos [Finkenzeller, 2010];
- datasheet da MIFARE Classic EV1, para organização de memória, autenticação e modelo de setores [NXP Semiconductors N.V., 2018];
- datasheet do MFRC522, para registradores, controle do leitor e integração com microcontroladores [NXP Semiconductors N.V., 2016];
- trabalhos de Garcia e de Koning Gans, para discussão acadêmica das limitações de segurança da MIFARE Classic [Garcia et al., 2008, de Koning Gans et al., 2008];

- documentação da família MIFARE Plus EV2, para contraste com soluções de geração mais recente [NXP Semiconductors, 2021].

2.15 Rodando na prática

O objetivo desta prática é executar um simulador de memória MIFARE Classic para visualizar autenticação, leitura, escrita e setor trailer sem depender de hardware físico.

2.15.1 Pré-requisitos

- Node.js 18 ou superior;
- terminal com suporte aos comandos do Node;
- diretório `docs/book/examples/cap2-rfid-mifare` disponível localmente.

2.15.2 Arquivos do laboratório

O mini projeto deste capítulo contém:

```
mifare-sim.mjs
```

2.15.3 Executando

No diretório do laboratório:

```
node mifare-sim.mjs
```

2.15.4 O que observar

Durante a execução, o aluno deve verificar:

- tentativa de leitura sem autenticação;
- autenticação do setor por meio da chave A;
- escrita de dados em um bloco comum;
- inspeção do setor trailer com chaves e bits de acesso.

2.15.5 Perguntas de análise

- Por que a leitura falha antes da autenticação, mas o trailer pode ser inspecionado?
- Em que sentido a chave padrão `FFFFFFFFFFFF` compromete a segurança da tag?
- O que impede um atacante de simplesmente copiar todos os blocos de uma tag legítima?
- Como a adição de um contador monotônico dificultaria um ataque de replay?

2.15.6 Extensão sugerida

Altere o simulador para:

- trocar a chave padrão do trailer e repetir a autenticação;
- reservar um bloco para armazenar um contador monotônico;
- adicionar uma verificação de integridade por *Message Authentication Code* (MAC).

2.15.7 Conexão com a arquitetura segura

Ao final da prática, o aluno deve ser capaz de explicar:

- por que a proteção baseada apenas em chaves estáticas é insuficiente;
- por que aplicações reais adicionam contador, desafio-resposta e verificação criptográfica fora da tag;
- como a camada de aplicação compensa limitações conhecidas da MIFARE Classic.

2.15.8 Localização do exemplo

Todos os arquivos usados nesta prática estão em:

`docs/book/examples/cap2-rfid-mifare`

Transição para o próximo capítulo. Este capítulo mostrou que a MIFARE Classic oferece uma organização de memória útil e um mecanismo de autenticação por setor, mas suas proteções nativas — chaves estáticas de 6 bytes e a cifra CRYPTO1 — não atendem, sozinhas, aos requisitos de segurança de sistemas modernos. A pergunta natural é: quais primitivas criptográficas uma aplicação pode adicionar sobre a tag para compensar essas limitações? O próximo capítulo responde a essa pergunta, explorando hash, HMAC, cifragem autenticada e assinaturas digitais — exatamente as ferramentas que o DIDGuard emprega sobre a camada RFID.

Capítulo 3

Fundamentos de Criptografia Aplicada

3.1 Objetivos de aprendizagem

Ao final deste capítulo, o aluno deve ser capaz de:

- distinguir integridade, autenticidade, confidencialidade e não repúdio;
- explicar o papel de funções hash, *Hash-based Message Authentication Code* (HMAC) e cifras autenticadas;
- descrever os conceitos básicos de criptografia simétrica e assimétrica;
- compreender o uso de curvas elípticas, chaves públicas e assinaturas digitais em sistemas distribuídos;
- justificar o uso de *nonce*, contador e mecanismos anti-replay em protocolos de acesso;
- relacionar esses fundamentos com a arquitetura de um sistema como o DIDGuard.

3.2 Propriedades de segurança fundamentais

Quatro propriedades aparecem com frequência em sistemas distribuídos:

- **Integridade**: garante que o conteúdo não foi alterado sem detecção.
- **Autenticidade**: fornece evidências de quem gerou ou aprovou a mensagem.
- **Confidencialidade**: protege o conteúdo contra leitura indevida.
- **Não repúdio**: dificulta que o autor de uma assinatura válida negue ter produzido aquela evidência.

Nem todo mecanismo entrega todas essas propriedades ao mesmo tempo. Uma função hash, por exemplo, ajuda na integridade, mas não oferece confidencialidade. Uma cifra simétrica protege confidencialidade, mas não substitui uma assinatura digital. Um bom projeto depende de combinar mecanismos adequados ao objetivo.

3.3 Funções hash criptográficas

Uma função hash criptográfica recebe uma entrada de tamanho arbitrário e produz uma saída de tamanho fixo, chamada de resumo ou digest. Na prática, algoritmos como a família *Secure Hash Algorithm* (SHA) são usados para condensar dados e permitir verificações rápidas de integridade [National Institute of Standards and Technology (NIST), 2015].

3.3.1 Propriedades esperadas

Uma função hash criptográfica moderna deve buscar, idealmente:

- **resistência à pré-imagem**: dado o hash, deve ser computacionalmente impraticável encontrar uma entrada que o gere;
- **resistência à segunda pré-imagem**: dada uma entrada, deve ser impraticável encontrar outra com o mesmo hash;
- **resistência a colisões**: deve ser impraticável encontrar duas entradas distintas que produzam o mesmo hash.

3.3.2 SHA-256

No contexto de aplicações modernas, o *Secure Hash Algorithm* 256 (SHA-256) é uma escolha comum para:

- resumir documentos;
- identificar conteúdo;
- construir provas de integridade;
- alimentar mecanismos como HMAC e derivação de material auxiliar.

Se dois sistemas calculam o mesmo SHA-256 sobre o mesmo conteúdo, eles devem obter exatamente o mesmo resultado. Essa previsibilidade é essencial para detecção de alteração e endereçamento por conteúdo.

3.4 Hash não é criptografia de sigilo

Um erro comum em projetos iniciantes é tratar hash como se fosse criptografia reversível. Hash não é feito para "descriptografar" depois. Ele é uma transformação unidirecional, usada principalmente para verificação, indexação e comparação segura de conteúdos.

Isso significa que:

- hash não substitui cifra quando o objetivo é manter um dado secreto;
- hash também não prova autoria por si só;
- para autenticar mensagens, é preciso combinar hash com chave secreta ou assinatura digital.

3.5 Código de autenticação de mensagem (MAC) e HMAC

Um *Message Authentication Code* (MAC) é um valor criptográfico calculado sobre uma mensagem com uso de chave secreta compartilhada. Seu objetivo é permitir que quem conhece a chave verifique integridade e autenticidade da mensagem.

Uma das construções mais difundidas é o *Hash-based Message Authentication Code* (HMAC), padronizado na *Request for Comments* (RFC) 2104 [Krawczyk et al., 1997]. Ele combina uma função hash com uma chave secreta de modo robusto e amplamente adotado.

3.5.1 Intuição de uso

Se duas partes compartilham a mesma chave secreta, ambas podem:

1. calcular o HMAC de uma mensagem;
2. transmitir mensagem e HMAC;
3. recomputar o HMAC no destino;
4. comparar os valores recebidos e calculados.

Se o valor divergir, algo está errado: a mensagem foi alterada, a chave não coincide ou a origem não é confiável.

3.5.2 Quando HMAC é útil

HMAC costuma ser uma boa escolha para:

- autenticação entre dispositivos e backend;
- proteção de contadores, estados e desafios;
- validação de dados gravados em memória local ou tag, quando há segredo compartilhado.

Por outro lado, HMAC não resolve cenários em que múltiplos verificadores precisam validar a mensagem sem conhecer a chave secreta. Nesses casos, assinaturas digitais são mais adequadas.

3.6 Criptografia simétrica

Na criptografia simétrica, a mesma chave, ou chaves diretamente relacionadas, é usada para cifrar e decifrar. O foco principal costuma ser confidencialidade.

O *Advanced Encryption Standard* (AES) é o padrão mais conhecido nessa categoria [National Institute of Standards and Technology (NIST), 2023b]. Ele é amplamente empregado em:

- proteção de dados sensíveis em repouso;
- canais de comunicação;
- cifras autenticadas como o modo *Galois/Counter Mode* (GCM).

3.6.1 AES-GCM

O modo *Galois/Counter Mode* (GCM) combina cifragem e autenticação em uma única construção. Em outras palavras, ele oferece confidencialidade e integridade/autenticidade do texto cifrado e de dados associados [Dworkin, 2007].

Esse tipo de construção é útil quando o sistema precisa:

- cifrar um conteúdo sensível;
- garantir que ele não foi alterado;
- rejeitar mensagens adulteradas antes de aceitar o resultado da decifragem.

Para funcionar corretamente, cifras autenticadas dependem de detalhes operacionais importantes, como:

- uso correto de chave;

- geração adequada de vetores de inicialização ou nonces;
- proibição de reutilização indevida de parâmetros críticos.

3.7 Criptografia assimétrica

Na criptografia assimétrica, cada entidade possui um par de chaves:

- **chave privada**: deve permanecer secreta e sob controle do titular;
- **chave pública**: pode ser distribuída a verificadores e serviços.

Esse modelo é central em identidades descentralizadas, assinatura de mensagens, autenticação de dispositivos e verificação por terceiros.

3.7.1 O que a assimetria permite

Com pares de chaves, um sistema pode:

- assinar dados com a chave privada;
- verificar assinaturas com a chave pública;
- estabelecer vínculos entre identidade lógica e material criptográfico;
- permitir que múltiplos verificadores validem evidências sem acesso ao segredo privado.

3.8 Curvas elípticas e ECDSA

Em muitos sistemas embarcados ou distribuídos, usa-se criptografia baseada em curvas elípticas por oferecer boa segurança com chaves menores que alternativas clássicas. O *Elliptic Curve Digital Signature Algorithm* (ECDSA) é um dos mecanismos mais conhecidos nesse espaço [National Institute of Standards and Technology (NIST), 2023a, Chen et al., 2023].

3.8.1 Por que curvas elípticas são populares

Elas costumam ser adotadas porque:

- reduzem custo de armazenamento de chaves;
- diminuem o volume de dados transmitidos;
- atendem bem a cenários com microcontroladores e dispositivos restritos.

3.8.2 Assinatura digital em alto nível

Em termos conceituais, o fluxo de assinatura digital é:

1. gerar um resumo do conteúdo ou da mensagem;
2. produzir uma assinatura com a chave privada;
3. transmitir conteúdo e assinatura;
4. verificar a assinatura com a chave pública correspondente.

Se a verificação for bem-sucedida, o verificador ganha evidências de que:

- o conteúdo não foi alterado desde a assinatura;
- a assinatura foi produzida por quem controla a chave privada associada.

Em implementações reais, o processo também depende de parâmetros como geração segura do valor efêmero da assinatura. Há inclusive padronização para uso determinístico desse valor em certos contextos [Pornin, 2013].

3.9 Nonce, contador e anti-replay

Mesmo quando integridade e autenticidade estão corretas, um sistema ainda pode falhar se aceitar repetição de mensagens antigas válidas. Esse é o problema de replay.

Dois mecanismos comuns para mitigar replay são:

- **nonce**: valor único, normalmente aleatório ou pseudoaleatório, usado uma única vez;
- **contador monotônico**: valor que aumenta a cada operação válida, impedindo reutilização de estados antigos.

3.9.1 Exemplo conceitual

Considere um pedido de autorização assinado ou protegido por HMAC. Se esse pedido puder ser retransmitido mais tarde exatamente igual, um atacante pode tentar reutilizá-lo. Ao incluir um nonce ou um contador esperado pelo verificador, o protocolo passa a distinguir uma mensagem nova de uma repetição indevida.

3.9.2 Requisitos de projeto

Para esses mecanismos funcionarem bem, é preciso:

- registrar se o nonce já foi usado;

- proteger o contador contra adulteração;
- sincronizar corretamente o estado entre as partes;
- rejeitar mensagens fora da janela esperada.

3.10 Derivação e gestão de chaves

Outro ponto central em criptografia aplicada é a gestão de chaves. Não basta escolher bons algoritmos; é preciso definir:

- onde a chave nasce;
- onde ela é armazenada;
- quem pode usá-la;
- quando ela expira ou deve ser rotacionada;
- como derivar chaves específicas para contextos diferentes.

Em muitos sistemas, uma chave-mestra não é usada diretamente em todas as operações. Em vez disso, o sistema deriva material específico por dispositivo, usuário, tag ou sessão. Isso reduz impacto de comprometimentos parciais e ajuda a compartimentar responsabilidades.

3.11 Aplicação desses conceitos em uma arquitetura como o DIDGuard

Em um sistema de controle de acesso com identidade descentralizada e dispositivos físicos, esses conceitos aparecem de forma concreta:

- **hash**: usado para resumir conteúdo, criar identificadores derivados e verificar integridade;
- **HMAC**: usado para proteger estados, desafios e dados associados a uma tag ou sessão;
- **criptografia simétrica**: usada para proteger dados sensíveis, como material biométrico cifrado antes do armazenamento;
- **criptografia assimétrica**: usada para identidade, assinatura e verificação de provas por terceiros;
- **nonce e contador**: usados para evitar replay em protocolos de autenticação;

- **gestão de chaves**: usada para separar segredos por dispositivo, tag, usuário ou função do sistema.

Essa combinação permite tratar a credencial física como apenas uma parte do protocolo, em vez de confiar cegamente que a segurança nativa da tag resolve sozinha o problema do acesso.

3.12 Erros comuns em projetos iniciantes

Alguns erros aparecem com frequência:

- usar apenas o identificador de uma tag como prova de identidade;
- armazenar segredo sensível sem cifragem adequada;
- confundir hash com criptografia reversível;
- reutilizar nonce ou contador sem controle;
- misturar a função de autenticação com a função de autorização;
- assumir que um algoritmo forte compensa uma gestão de chaves fraca.

Criptografia aplicada é tanto sobre algoritmos quanto sobre protocolo, implementação, armazenamento e operação.

3.13 Leituras recomendadas

- *Federal Information Processing Standards* (FIPS) 180-4, para funções hash da família SHA [National Institute of Standards and Technology (NIST), 2015];
- RFC 2104, para fundamentos e construção de HMAC [Krawczyk et al., 1997];
- FIPS 197, para o algoritmo AES [National Institute of Standards and Technology (NIST), 2023b];
- *National Institute of Standards and Technology* (NIST) SP 800-38D, para AES-GCM e *Galois Message Authentication Code* (GMAC) [Dworkin, 2007];
- FIPS 186-5, para assinatura digital [National Institute of Standards and Technology (NIST), 2023a];
- NIST SP 800-186, para parâmetros de curvas elípticas [Chen et al., 2023];
- RFC 6979, para uso determinístico em assinaturas ECDSA [Pornin, 2013].

3.14 Rodando na prática

O objetivo desta prática é executar um laboratório autocontido de criptografia aplicada para observar, em um único fluxo, hash, HMAC, cifragem autenticada e assinatura digital.

3.14.1 Pré-requisitos

- Node.js 18 ou superior;
- terminal com suporte aos comandos do Node;
- diretório `docs/book/examples/cap3-crypto-aplicada` disponível localmente.

3.14.2 Arquivos do laboratório

O mini projeto deste capítulo contém:

`crypto-lab.mjs`

3.14.3 Executando

No diretório do laboratório:

```
node crypto-lab.mjs
```

3.14.4 O que observar

- o *digest* SHA-256 produzido a partir de uma mensagem simples;
- a diferença entre o valor do hash e o valor do HMAC;
- a cifragem e a decifragem com `AES-256-GCM`;
- a validação de uma assinatura ECDSA para a mensagem original e a falha para a mensagem alterada.

3.14.5 Perguntas de análise

- Em que cenário um hash sozinho é suficiente?
- Em que cenário é necessário um HMAC?
- O que a `authTag` do modo GCM protege?
- Por que a verificação de assinatura falha quando a mensagem é alterada?

3.14.6 Extensão sugerida

Altere o laboratório para:

- usar um *nonce* fixo e discutir o risco resultante;
- substituir a mensagem por um pequeno vetor serializado em JSON;
- gerar uma segunda assinatura com outro par de chaves e comparar os resultados.

3.14.7 Localização do exemplo

Todos os arquivos usados nesta prática estão em:

`docs/book/examples/cap3-crypto-aplicada`

Transição para o próximo capítulo. As primitivas apresentadas neste capítulo — hash, HMAC, cifragem autenticada e assinaturas digitais — permitem proteger dados, autenticar mensagens e verificar identidades. Porém, resta uma questão-chave: onde ficam as regras de acesso e como elas podem ser verificadas de forma auditável por múltiplas partes? O próximo capítulo introduz blockchain e contratos inteligentes como camada verificável de política, mostrando como registrar, consultar e auditar permissões de acesso em um ambiente onde nenhum participante isolado controla a verdade.

Capítulo 4

Blockchain e Contratos Inteligentes

4.1 Objetivos de aprendizagem

Ao final deste capítulo, o aluno deve ser capaz de:

- explicar o que caracteriza uma blockchain e por que ela difere de um banco de dados convencional;
- distinguir bloco, transação, estado, nó, consenso e contrato inteligente;
- compreender o papel da *Ethereum Virtual Machine* (EVM), do custo computacional e do armazenamento on-chain;
- interpretar o papel de eventos, funções de leitura e funções de escrita em contratos;
- justificar quando vale a pena usar blockchain em uma arquitetura de identidade e controle de acesso.

4.2 Do livro-razão distribuído ao contrato inteligente

Uma blockchain pode ser entendida como um livro-razão distribuído no qual múltiplos nós mantêm uma visão compartilhada e verificável de transações e estado. Em vez de confiar em uma única autoridade central para armazenar e validar registros, a rede aplica regras de consenso e validação criptográfica sobre cada alteração [Nakamoto, 2008, Buterin, 2014].

Embora o tema tenha ganhado popularidade com criptomoedas, o conceito relevante para este curso é mais amplo: blockchain é uma infraestrutura de registro verificável, com forte rastreabilidade e regras de atualização definidas por protocolo.

4.3 Elementos básicos de uma blockchain

Os componentes conceituais mais importantes são:

- **bloco**: agrupamento ordenado de transações e metadados;
- **transação**: operação solicitada por um participante da rede;
- **nó**: instância de software que valida, propaga ou armazena o estado da rede;
- **estado**: fotografia lógica atual dos dados relevantes para execução do protocolo;
- **consenso**: mecanismo pelo qual a rede concorda sobre a ordem e validade das alterações;
- **hash do bloco**: referência criptográfica que vincula blocos sucessivos.

Esse encadeamento por hash dificulta alteração retroativa invisível, porque uma mudança em um bloco compromete os vínculos criptográficos dos blocos seguintes.

4.4 Por que blockchain não é apenas banco de dados

Um banco de dados convencional costuma oferecer:

- alta flexibilidade de consulta;
- controle administrativo centralizado;
- atualização direta por operadores autorizados.

Uma blockchain, por sua vez, prioriza:

- verificabilidade compartilhada;
- histórico rastreável de transações;
- execução de regras padronizadas em ambiente distribuído;
- resistência a adulteração não consensual.

Essa diferença é essencial. Blockchain não substitui qualquer banco de dados; ela faz sentido quando auditoria, confiança distribuída, política verificável e integridade histórica são requisitos relevantes.

4.5 Transações, contas e estado

Em plataformas como Ethereum, a rede opera sobre um modelo de estado global. Cada bloco confirma transações que alteram esse estado. Em termos didáticos, uma transação pode:

- transferir valor;
- criar um contrato;
- invocar uma função de contrato;
- registrar um evento como efeito observável da execução.

As contas geralmente se dividem em:

- **contas externas**: controladas por chaves privadas de usuários ou serviços;
- **contas de contrato**: controladas pelo código do contrato implantado na rede.

4.6 Consenso em alto nível

O consenso é o mecanismo que permite à rede decidir qual é a próxima versão válida do livro-razão distribuído. O aluno não precisa dominar, neste momento, cada detalhe de implementação de prova de trabalho ou prova de participação. O ponto essencial é entender que:

- diferentes redes usam diferentes mecanismos de consenso;
- o consenso afeta desempenho, custo, segurança e finalização;
- a aplicação deve respeitar o tempo de confirmação e o modelo de confiança da rede usada.

Em ambientes didáticos, também é comum usar redes locais de desenvolvimento, nas quais a semântica do contrato é preservada, mas o consenso é simplificado para facilitar testes.

4.7 Ethereum e a Ethereum Virtual Machine

Ethereum se tornou uma das plataformas mais importantes para contratos inteligentes ao introduzir um ambiente geral de execução programável. Seu modelo permite armazenar estado e executar lógica determinística em todos os nós validadores relevantes da rede [Buterin, 2014, Wood, 2025].

A *Ethereum Virtual Machine* (EVM) é a máquina abstrata que executa o bytecode dos contratos. Conceitualmente, ela:

- recebe instruções do contrato compilado;
- processa operações aritméticas, lógicas e de armazenamento;
- mantém um contexto de execução padronizado;
- garante que a execução seja determinística para todos os participantes.

Esse determinismo é crítico. Se dois nós executarem a mesma transação sobre o mesmo estado e chegarem a resultados diferentes, a rede perde consistência.

4.8 Gas e custo computacional

Em Ethereum e plataformas similares, a execução não é tratada como custo zero. Cada operação consome *gas*, uma unidade abstrata associada ao esforço computacional e ao uso de recursos da rede [Wood, 2025, Ethereum Foundation, 2026].

4.8.1 Por que gas existe

O mecanismo de gas existe para:

- limitar execução infinita ou abusiva;
- cobrar pelo uso de armazenamento e computação compartilhada;
- incentivar contratos mais eficientes.

4.8.2 Impacto no desenho de contratos

Isso leva a consequências práticas:

- escrever em armazenamento custa mais do que ler;
- laços longos e estruturas desnecessárias aumentam custo;
- nem tudo deve ser colocado on-chain;
- cálculos pesados e grandes volumes de dados tendem a ser melhores fora da blockchain.

4.9 Contratos inteligentes

Um contrato inteligente é um programa armazenado na blockchain e executado segundo regras determinísticas da plataforma. Apesar do nome, ele não é “inteligente” no sentido humano; trata-se de código com regras explícitas.

Em uma aplicação didática de controle de acesso, um contrato pode ser responsável por:

- registrar o vínculo entre uma identidade e um identificador de referência;
- armazenar permissões ou janelas temporais de acesso;
- validar se um dispositivo está autorizado a consultar determinado estado;
- emitir eventos que possam ser auditados por outros componentes.

4.10 Leitura, escrita e eventos

Ao estudar contratos, vale separar três categorias:

- **funções de leitura**: consultam estado sem alterá-lo;
- **funções de escrita**: alteram estado e exigem transação;
- **eventos**: registros emitidos pelo contrato para facilitar observação externa.

4.10.1 Funções de leitura

São úteis para:

- consultar permissões atuais;
- recuperar apontadores ou identificadores vinculados;
- depurar o comportamento esperado do contrato.

4.10.2 Funções de escrita

São usadas quando a aplicação precisa:

- registrar novo estado;
- revogar ou conceder acesso;
- atualizar vínculos e políticas.

Como escrevem estado, essas funções têm custo, geram transações e exigem confirmação.

4.10.3 Eventos

Eventos não substituem armazenamento. Eles servem principalmente para:

- notificar sistemas externos;
- facilitar indexação e monitoramento;
- apoiar trilhas de auditoria.

Uma aplicação cliente ou um relayer pode escutar eventos e reagir a eles, mas a lógica principal de autorização não deve depender apenas de suposições implícitas sobre histórico de eventos se o estado atual pode ser consultado diretamente.

4.11 Estruturas típicas em contratos

Ao modelar contratos, é comum encontrar:

- variáveis de estado;
- *mappings* entre identificadores e registros;
- estruturas compostas para armazenar atributos de autorização;
- verificações de permissões por remetente, papel ou contexto;
- eventos emitidos após mudanças relevantes.

Em termos de projeto, uma boa modelagem tenta equilibrar:

- simplicidade de verificação;
- custo on-chain;
- clareza de auditoria;
- separação entre dados mínimos on-chain e dados ricos off-chain.

4.12 Armazenamento on-chain versus off-chain

Nem todo dado deve ficar na blockchain. Há pelo menos três razões para isso:

- **custo**: armazenar muitos dados on-chain é caro;
- **privacidade**: alguns dados não devem ser públicos;
- **imutabilidade**: uma vez publicado, o dado on-chain é difícil de remover.

Por isso, arquiteturas modernas frequentemente colocam on-chain apenas:

- identificadores;
- hashes;
- apontadores;
- estados mínimos de autorização;
- evidências necessárias para verificação.

Os dados mais extensos ou sensíveis costumam ficar fora da blockchain, em serviços auxiliares ou armazenamentos distribuídos. Esse será o gancho do próximo capítulo sobre *InterPlanetary File System* (IPFS).

4.13 Por que blockchain faz sentido em controle de acesso

Em um sistema didático de identidade e autorização, blockchain pode ser útil quando o objetivo é:

- registrar regras de acesso com trilha auditável;
- permitir verificação independente do estado autorizado;
- separar a política verificável da interface de administração;
- reduzir dependência de um único banco central opaco para decisão crítica.

Ela não elimina a necessidade de outros componentes. Firmware, relayer, frontend, armazenamento off-chain e gestão de chaves continuam sendo partes essenciais da arquitetura.

4.14 Limitações e trade-offs

Uma decisão madura sobre blockchain precisa reconhecer seus custos:

- latência maior do que sistemas puramente locais;
- custo financeiro ou computacional por transação;
- dificuldade de corrigir rapidamente dados mal modelados;
- exposição pública de parte do estado, conforme a rede usada;

- necessidade de desenhar com cuidado o que fica on-chain e o que fica fora.

Em outras palavras, blockchain não deve ser usada por moda. Ela deve ser escolhida quando os benefícios de verificabilidade, rastreabilidade e execução compartilhada superam seus custos operacionais.

4.15 Relação com uma arquitetura como a do DIDGuard

Em uma arquitetura inspirada no DIDGuard, o contrato inteligente tende a assumir o papel de camada verificável de política e vínculo mínimo. Isso significa que ele pode:

- associar um identificador descentralizado a um apontador externo;
- manter autorização por dispositivo, identidade ou janela temporal;
- fornecer funções de consulta de estado atual;
- emitir eventos úteis para observação do sistema.

Enquanto isso:

- o firmware lida com a credencial física e a coleta local;
- o relayer integra chamadas ao contrato e serviços auxiliares;
- o armazenamento distribuído hospeda dados mais ricos;
- o frontend fornece operação e visualização.

Essa divisão de responsabilidades é importante porque evita transformar o contrato em um repositório de tudo. O contrato deve guardar o que precisa ser público, verificável e mínimo.

4.16 Leituras recomendadas

- white paper do Bitcoin, para entender a noção original de livro-razão distribuído e encadeamento por hash [Nakamoto, 2008];
- white paper do Ethereum, para motivação da plataforma programável [Buterin, 2014];
- *Ethereum Yellow Paper*, para visão mais formal da execução e do custo computacional [Wood, 2025];

- documentação oficial de Solidity, para modelagem de contratos, tipos, eventos e boas práticas [Solidity Team, 2026];
- documentação de opcodes da EVM, para aprofundamento no modelo de execução [Ethereum Foundation, 2026].

4.17 Rodando na prática

O objetivo desta prática é implantar um contrato simples em uma blockchain local, gravar um estado de autorização e observar a diferença entre leitura, escrita e evento emitido.

4.17.1 Pré-requisitos

- Node.js 18 ou superior;
- terminal com suporte a `npm` e `npx`;
- diretório `docs/book/examples/cap4-blockchain-contratos` disponível localmente.

4.17.2 Arquivos do laboratório

O mini projeto deste capítulo contém:

```
contracts/AccessRegistry.sol
scripts/deploy.js
scripts/demo.js
hardhat.config.js
package.json
```

4.17.3 Instalação

No diretório do laboratório:

```
npm install
```

4.17.4 Execução

Em um terminal:

```
npx hardhat node
```

Em outro terminal:

```
npx hardhat run scripts/deploy.js --network localhost
```

Copie o endereço do contrato implantado e execute:

```
$env:ACCESS_REGISTRY_ADDRESS="COLE_O_ENDERECO_AQUI"
npx hardhat run scripts/demo.js --network localhost
```

Alternativamente, para executar deploy e demonstração em um único passo (sem copiar endereços):

```
npx hardhat run scripts/demo-auto.js --network localhost
```

4.17.5 O que observar

- o contrato sendo implantado na rede local;
- a chamada de escrita `setAccess` gerando uma transação;
- a chamada de leitura `getAccess` retornando o estado sem criar nova transação;
- o evento `AccessUpdated` sendo recuperado do recibo.

4.17.6 Perguntas de análise

- Por que a leitura pode ser modelada como `view`?
- Por que a escrita precisa de uma transação assinada e minerada?
- Qual informação foi mantida on-chain e qual informação ficou fora do contrato?

4.17.7 Extensão sugerida

Evolua o contrato para:

- registrar o identificador de um dispositivo;
- armazenar um tempo de expiração;
- impedir atualizações com prazo já expirado.

4.17.8 Localização do exemplo

Todos os arquivos usados nesta prática estão em:

`docs/book/examples/cap4-blockchain-contratos`

Encerramento e próximos passos. Com este capítulo, a apostila cobriu as quatro camadas fundamentais do DIDGuard: identidade descentralizada (DIDs), credencial física (RFID/MIFARE), primitivas criptográficas e registro verificável de permissões (blockchain). Os capítulos seguintes — quando disponíveis — abordarão armazenamento distribuído com IPFS, firmware do ESP32, integração via relayer e frontend de operação, completando o percurso da teoria à implementação.

Capítulo 5

ESP32, ESP-IDF e PlatformIO

5.1 Objetivos de aprendizagem

Ao final deste capítulo, o aluno deve ser capaz de:

- explicar o papel do ESP32 como plataforma de borda em um sistema de identidade e controle de acesso;
- distinguir hardware, framework oficial ESP-IDF e camada de automação fornecida pelo PlatformIO;
- compreender a estrutura básica de um projeto ESP-IDF em PlatformIO;
- configurar conectividade Wi-Fi em modo estação;
- inicializar o barramento SPI e conectar um leitor MFRC522;
- descrever como primitivas criptográficas locais podem apoiar a geração de um *DID Document*.

5.2 Por que o firmware merece um capítulo próprio

Em uma arquitetura como a do DIDGuard, o firmware é a camada que toca o mundo físico. É nele que a aplicação:

- lê uma credencial em uma tag;
- estabelece conectividade de rede;
- protege material criptográfico;
- produz evidências locais que serão consumidas por serviços externos.

Essa posição faz do firmware uma peça central. Um erro nessa camada pode comprometer disponibilidade, segurança e confiabilidade de todo o sistema.

5.3 Visão geral do ESP32

ESP32 é uma família de sistemas em chip (*System on Chip*, SoC) voltada para aplicações embarcadas conectadas. A plataforma combina processamento, memória, periféricos e interfaces de comunicação em um único chip [Espressif Systems, 2026f].

Do ponto de vista de projeto, o ESP32 é atraente porque reúne:

- conectividade Wi-Fi integrada;
- suporte a múltiplos periféricos seriais, incluindo SPI, I^2C e UART;
- recursos suficientes para lógica local, buffers, serialização e operações criptográficas;
- ecossistema maduro de ferramentas e documentação.

Em termos didáticos, ele permite que o aluno trabalhe simultaneamente com rede, periféricos, concorrência e segurança em um único ambiente de desenvolvimento.

5.4 ESP-IDF: o framework oficial

ESP-IDF (*Espressif IoT Development Framework*) é o framework oficial de desenvolvimento para a família ESP32 [Espressif Systems, 2026c]. Ele organiza a aplicação em componentes e fornece:

- APIs para Wi-Fi, rede, armazenamento, tarefas e sincronização;
- drivers de periféricos como SPI, GPIO e timers;
- integração com Mbed TLS para operações criptográficas;
- sistema de build baseado em CMake;
- configuração detalhada por Kconfig e `sdkconfig`.

O ESP-IDF não é apenas uma biblioteca. Ele define convenções de organização, inicialização e dependências entre componentes, o que impacta diretamente a forma como o firmware é projetado.

5.5 PlatformIO como camada de produtividade

PlatformIO não substitui o ESP-IDF. Ele automatiza tarefas comuns do fluxo de desenvolvimento, como instalação de toolchain, resolução de dependências, build, upload e monitoramento serial [PlatformIO, 2026a,b].

Na prática, o aluno passa a trabalhar com dois níveis:

- **ESP-IDF**: API, componentes e semântica do firmware;
- **PlatformIO**: organização do projeto, ambientes de build e automação de ferramentas.

Essa distinção é importante. Quando algo falha, o aluno precisa saber se o problema está na lógica do firmware, na configuração do framework ou no ambiente de build.

5.6 Estrutura típica de projeto

Um projeto ESP-IDF via PlatformIO tende a conter, no mínimo:

```
platformio.ini
sdkconfig.defaults
CMakeLists.txt
include/
src/
```

O papel desses artefatos pode ser resumido assim:

- `platformio.ini`: define placa, framework, monitor serial e opções de build;
- `sdkconfig.defaults`: registra configurações iniciais do menuconfig;
- `CMakeLists.txt`: conecta o projeto ao sistema de build do ESP-IDF;
- `include/`: cabeçalhos públicos da aplicação;
- `src/`: código-fonte principal e arquivos auxiliares.

5.7 Configuração via `platformio.ini`

Um exemplo mínimo de configuração pode ser:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
monitor_speed = 115200
build_flags =
  -D WIFI_SSID=\"LAB_WIFI\"
  -D WIFI_PASS=\"LAB_PASS\"
```

Esse arquivo concentra decisões operacionais importantes:

- qual placa será alvo do build;

- qual framework será usado;
- quais macros de compilação a aplicação receberá;
- como o monitor serial será aberto.

5.8 Configuração persistente via `sdkconfig.defaults`

O arquivo `sdkconfig.defaults` permite registrar preferências que devem existir desde o primeiro build. Um exemplo didático:

```
CONFIG_ESPTOOLPY_FLASHSIZE_4MB=y
CONFIG_ESP_MAIN_TASK_STACK_SIZE=8192
CONFIG_ESP_WIFI_ENABLED=y
CONFIG_MBEDTLS_PSA_CRYPTO_CLIENT=y
```

Nem toda configuração precisa estar nesse arquivo, mas ele é útil para manter reprodutibilidade entre máquinas e evitar que parâmetros críticos dependam exclusivamente do estado local do menuconfig.

5.9 Inicialização básica do sistema

Antes de usar Wi-Fi ou periféricos, o firmware costuma inicializar:

- NVS (*Non-Volatile Storage*);
- interface de rede (`esp_netif`);
- laço de eventos do sistema;
- drivers ou componentes necessários ao fluxo da aplicação.

No caso do Wi-Fi, a própria documentação do ESP-IDF deixa claro que a pilha depende dessa ordem de inicialização [Espressif Systems, 2026e,b].

5.10 Exemplo: conexão Wi-Fi em modo estação

O modo estação (*Station Mode*, STA) é o cenário em que o ESP32 se conecta a um ponto de acesso existente. Em aplicações como o DIDGuard, isso é suficiente para:

- consultar um relayer;
- enviar leituras de tags;

- requisitar atualização de permissões;
- obter materiais auxiliares, como um documento ou um desafio criptográfico.

Um trecho típico de inicialização em ESP-IDF é:

```
ESP_ERROR_CHECK(nvs_flash_init());
ESP_ERROR_CHECK(esp_netif_init());
ESP_ERROR_CHECK(esp_event_loop_create_default());
esp_netif_create_default_wifi_sta();

wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
ESP_ERROR_CHECK(esp_wifi_init(&cfg));
```

Depois disso, a aplicação define credenciais e inicia a conexão:

```
wifi_config_t wifi_config = {
  .sta = {
    .ssid = WIFI_SSID,
    .password = WIFI_PASS,
    .threshold.authmode = WIFI_AUTH_WPA2_PSK
  }
};

ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
ESP_ERROR_CHECK(esp_wifi_start());
ESP_ERROR_CHECK(esp_wifi_connect());
```

Do ponto de vista didático, esse exemplo reforça três ideias:

- a aplicação depende de inicialização ordenada de subsistemas;
- Wi-Fi é orientado a eventos, e não apenas a chamadas sequenciais;
- a conectividade deve ser tratada como parte da lógica de aplicação, inclusive com reconexão e timeout.

5.11 Integração com o RC522 via SPI

O MFRC522 se comunica com o microcontrolador por SPI. No lado do ESP32, isso significa:

- configurar os pinos do barramento;

- inicializar o *host* SPI;
- criar um driver de transporte para o leitor;
- delegar a lógica de polling, seleção e estados da PICC a uma biblioteca especializada [NXP Semiconductors N.V., 2016, Espressif Systems, 2026d].

No contexto deste material, a forma preferida de integração não é manipular registradores do MFRC522 diretamente. O exemplo do capítulo usa a biblioteca `abobija/rc522` como componente gerenciado do ESP-IDF, declarada em `idf_component.yml` [Bobija, 2026]. Essa biblioteca encapsula:

- criação do transporte SPI;
- instalação do driver do leitor;
- polling periódico de tags;
- transições de estado da PICC;
- APIs de nível mais alto para leitura, autenticação e escrita.

Um exemplo de configuração do driver SPI da biblioteca é:

```
rc522_spi_config_t driver_config = {
  .host_id = SPI3_HOST,
  .bus_config = &(spi_bus_config_t){
    .miso_io_num = 25,
    .mosi_io_num = 23,
    .sclk_io_num = 19,
    .max_transfer_sz = 32
  },
  .dev_config = {
    .clock_speed_hz = 1000000,
    .spics_io_num = 22,
  },
  .rst_io_num = -1
};
```

Depois disso, o scanner é criado e iniciado no nível de componente:

```
rc522_spi_create(&driver_config, &driver);
rc522_driver_install(driver);

rc522_config_t scanner_config = {
```

```
  .driver = driver,
  .poll_interval_ms = 250
};

rc522_create(&scanner_config, &scanner);
rc522_register_events(scanner,
                      RC522_EVENT_PICC_STATE_CHANGED,
                      on_picc_state_changed,
                      NULL);
rc522_start(scanner);
```

5.11.1 Por que usar a biblioteca como componente

Essa abordagem é preferível em um firmware real porque separa responsabilidades:

- o componente `abobija/rc522` cuida do protocolo do leitor e do ciclo de vida das tags;
- a aplicação foca no que fazer quando uma tag aparece, é autenticada ou precisa ser lida;
- a lógica de domínio, como geração de DID, MAC, contador e assinatura, fica desacoplada do detalhe elétrico do MFRC522.

5.11.2 Acesso direto versus componente

Vale separar claramente dois estilos de integração:

- **acesso direto a registradores**: útil para estudo de baixo nível, depuração de SPI e entendimento da interface do chip;
- **uso de componente**: mais adequado para aplicações reais, porque reduz código repetitivo, melhora manutenção e reaproveita uma API já testada.

Para um capítulo didático introdutório, é válido conhecer o registrador e o barramento. Para um projeto funcional, a barra técnica correta é subir um nível e trabalhar com o componente.

5.12 Criptografia local no ESP32

O ESP-IDF integra Mbed TLS, o que permite realizar operações criptográficas no próprio dispositivo [Espressif Systems, 2026a]. Em um projeto como o DIDGuard, isso é relevante para:

- gerar ou carregar chaves assimétricas;
- calcular hashes;
- assinar desafios;
- derivar identificadores e evidências locais.

5.12.1 Geração de chave elíptica

Uma forma didática de gerar uma chave é:

```
mbedtls_pk_init(&pk);
mbedtls_pk_setup(&pk, mbedtls_pk_info_from_type(MBEDTLS_PK_ECKEY));
mbedtls_ecp_gen_key(MBEDTLS_ECP_DP_SECP256R1,
                    mbedtls_pk_ec(pk),
                    mbedtls_ctr_drbg_random,
                    &ctr_drbg);
```

Esse fluxo cria uma chave baseada na curva P-256, que é suficiente para experimentos de identidade e assinatura digital em ambiente embarcado.

5.12.2 Derivação de um identificador

Uma estratégia comum é derivar um identificador estável a partir da chave pública ou de um material público do dispositivo. Um exemplo didático:

```
mbedtls_sha256(pubkey_bytes, pubkey_len, digest, 0);
```

O resultado pode ser convertido para hexadecimal e usado como parte do identificador:

```
did:esp32:4f6a8b...
```

Em uma implementação real, a semântica exata do método DID precisa ser definida explicitamente. Aqui, a ideia central é mostrar como o firmware pode contribuir para produzir um identificador verificável.

5.13 Montando um DID Document no firmware

Depois de gerar a chave e derivar o identificador, a aplicação pode serializar um documento JSON com os campos mínimos necessários para representar a identidade do dispositivo.

Um exemplo didático de estrutura:

```
{
  "@context": ["https://www.w3.org/ns/did/v1"],
  "id": "did:esp32:4f6a8b...",
  "verificationMethod": [{
    "id": "did:esp32:4f6a8b...#device-key-1",
    "type": "EcdsaSecp256r1VerificationKey2019",
    "controller": "did:esp32:4f6a8b...",
    "publicKeyHex": "04ABCD..."
  }]
}
```

Neste exemplo, o campo `publicKeyHex` é usado por simplicidade didática e aparece nas registries associadas ao ecossistema DID, embora outras representações como JWK ou Multibase também sejam comuns [World Wide Web Consortium (W3C), 2024].

Neste ponto, a intenção do firmware não é resolver todo o ecossistema de identidade. Seu papel é preparar dados confiáveis para que camadas superiores façam persistência, distribuição e verificação.

5.14 Fluxo conceitual completo

Uma sequência coerente para o firmware pode ser descrita assim:

1. inicializar NVS, rede e laço de eventos;
2. conectar o ESP32 ao Wi-Fi;
3. inicializar o barramento SPI e o leitor RC522;
4. ler a tag e extrair dados relevantes;
5. gerar ou carregar chave criptográfica do dispositivo;
6. construir um DID e serializar o *DID Document*;
7. entregar esse material a uma camada superior por HTTP, serial ou outro canal definido pela arquitetura.

5.15 Boas práticas de projeto

Ao elaborar firmware para um caso como este, convém observar:

- separar drivers, serviços e lógica de aplicação;
- evitar segredos em texto puro no código-fonte;

- centralizar configuração de pinos, credenciais e parâmetros;
- registrar logs úteis para depuração sem vazar informação sensível;
- tratar falhas de rede, autenticação e periféricos como parte normal do fluxo.

5.16 Leituras recomendadas

- documentação geral do ESP-IDF para visão de componentes, build e APIs [Espressif Systems, 2026c];
- referência de Wi-Fi do ESP-IDF, para modos de operação, eventos e sequência de inicialização [Espressif Systems, 2026e];
- documentação de NVS, para entender persistência local e dependências da pilha de rede [Espressif Systems, 2026b];
- documentação do driver SPI Master, para modelagem correta do barramento e de transações [Espressif Systems, 2026d];
- documentação do Mbed TLS no ESP-IDF, para geração de chaves e operações criptográficas embarcadas [Espressif Systems, 2026a];
- documentação do PlatformIO para uso do framework `espidf` e do ecossistema `espressif32` [PlatformIO, 2026a,b];
- documentação do componente `abobija/rc522`, para API de eventos, criação do driver e exemplos de uso [Bobija, 2026];
- página oficial do ESP32, para posicionamento da plataforma e visão geral de recursos [Espressif Systems, 2026f].

5.17 Rodando na prática

O objetivo desta prática é estudar um mini projeto autocontido que mostra a organização de um firmware em PlatformIO com ESP-IDF, incluindo Wi-Fi, SPI/RC522 e geração de um *DID Document* simplificado.

5.17.1 Pré-requisitos

- PlatformIO instalado;
- toolchain do ambiente `espressif32` disponível;
- diretório `docs/book/examples/cap5-firmware-esp32` disponível localmente.

5.17.2 Arquivos do laboratório

O mini projeto deste capítulo contém:

```
platformio.ini
sdkconfig.defaults
CMakeLists.txt
src/CMakeLists.txt
src/idf_component.yml
src/main.c
include/lab_config.h
```

5.17.3 O que o aluno deve observar

- como o `platformio.ini` descreve o ambiente de build;
- como o `sdkconfig.defaults` antecipa escolhas de configuração;
- como o `idf_component.yml` declara a dependência `abobija/rc522`;
- como a inicialização do Wi-Fi depende de NVS, `esp_netif` e laço de eventos;
- como a biblioteca RC522 cria o driver SPI e publica eventos de mudança de estado da tag;
- como a aplicação gera uma chave local, deriva um identificador e serializa um documento JSON.

5.17.4 Execução sugerida

No diretório do laboratório:

```
pio run
pio run -t upload
pio device monitor
```

5.17.5 Resultados esperados

O aluno deve encontrar no log serial mensagens equivalentes a:

- inicialização bem-sucedida do Wi-Fi;
- inicialização do driver e do scanner RC522;
- detecção de uma tag pelo evento `RC522_EVENT_PICC_STATE_CHANGED`;
- impressão do DID gerado e do JSON associado.

5.17.6 Extensão sugerida

Como exercício, o aluno pode:

- substituir macros de SSID e senha por leitura de NVS;
- mover a lógica do RC522 para um módulo dedicado;
- enviar o *DID Document* por HTTP para um endpoint local;
- assinar um desafio simples com a chave gerada no firmware.

5.17.7 Localização do exemplo

Todos os arquivos usados nesta prática estão em:

`docs/book/examples/cap5-firmware-esp32`

Glossário do Capítulo 1

DID	Identificador descentralizado usado para representar um sujeito de forma verificável e independente de uma autoridade central única.
DID Document	Documento estruturado que descreve os metadados associados a um DID, incluindo chaves, métodos de autenticação e serviços.
Método DID	Conjunto de regras que define como um DID é criado, resolvido, atualizado e, quando aplicável, desativado.
Resolução de DID	Processo de obter o DID Document e os metadados associados a partir de um DID.
Dereferenciamento	Processo de obter um recurso específico associado ao DID, como uma chave identificada por fragmento ou um serviço declarado.
Resolver	Componente que interpreta o método DID e coordena a obtenção do DID Document.
Driver do método	Implementação específica das regras de resolução de um determinado método DID.
Fonte de verdade	Infraestrutura onde o estado do DID está efetivamente registrado, como um servidor web, blockchain ou registro distribuído.
W3C	Sigla de *World Wide Web Consortium*, organização responsável por diversos padrões da web, incluindo o DID Core.
Controller	Entidade que controla o DID ou um método de verificação associado a ele.

Verification Method Mecanismo criptográfico declarado no DID Document para verificação de assinaturas, autenticação ou outros usos.

Authentication Propriedade do DID Document que indica quais métodos podem ser usados para autenticar o sujeito.

Assertion Method Propriedade que indica quais métodos podem ser usados para assinar afirmações, como credenciais ou declarações verificáveis.

Key Agreement Propriedade que indica métodos aptos a estabelecer segredos compartilhados para comunicação segura.

Service Entrada do DID Document usada para anunciar endpoints ou recursos associados à identidade.

Service Endpoint Endereço do serviço associado a uma entrada de serviço no DID Document.

JSON Sigla de *JavaScript Object Notation*, formato textual amplamente usado para troca e representação estruturada de dados.

JWK Sigla de *JSON Web Key*, formato baseado em JSON para representação de chaves criptográficas.

JWS Sigla de *JSON Web Signature*, formato e conjunto de convenções usados para representar assinaturas digitais em estruturas JSON.

EC Sigla de *Elliptic Curve*, usada para indicar o emprego de criptografia baseada em curvas elípticas.

HTTP Sigla de *Hypertext Transfer Protocol*, protocolo de aplicação usado na comunicação entre clientes e servidores web.

HTTPS Sigla de *Hypertext Transfer Protocol Secure*, versão protegida do HTTP com uso de criptografia em trânsito.

DHT Sigla de *Distributed Hash Table*, estrutura distribuída usada para descoberta e organização de informações em redes descentralizadas.

IPFS Sigla de *InterPlanetary File System*, sistema distribuído de armazenamento endereçado por conteúdo.

CID Sigla de *Content Identifier*, identificador de conteúdo usado em sistemas como IPFS para localizar dados a partir do hash do conteúdo.

API Sigla de interface de programação de aplicações, usada para expor operações e serviços entre sistemas.

RFC Sigla de *Request for Comments*, série de documentos técnicos usados para padronização de protocolos e tecnologias da internet.

URI Sigla de *Uniform Resource Identifier*, identificador padronizado usado para nomear e localizar recursos na web e em outros sistemas.

Glossário do Capítulo 2

RFID Sigla de *Radio Frequency Identification*, tecnologia de identificação por radiofrequência usada para detectar e trocar dados com etiquetas, cartões e dispositivos sem contato físico.

HF Sigla de alta frequência, faixa de operação em 13,56 MHz usada por cartões e leitores sem contato da família ISO/IEC 14443.

PICC Sigla de *Proximity Integrated Circuit Card*, termo usado para a tag, cartão ou credencial sem contato energizada pelo campo do leitor.

PCD Sigla de *Proximity Coupling Device*, termo usado para o leitor que energiza a tag e conduz a comunicação sem contato.

UID Sigla de *Unique Identifier*, identificador de fábrica usado para distinguir uma tag no processo de seleção.

NUID Sigla de *Non-Unique Identifier*, identificador não único usado por algumas variantes de cartões de 4 bytes compatíveis com a família MIFARE.

Setor Agrupamento lógico de blocos dentro da memória de uma MIFARE Classic, usado para organizar dados e definir permissões.

Bloco Unidade básica de armazenamento na MIFARE Classic, com 16 bytes por bloco.

Bloco de fabricante Primeiro bloco do setor 0, reservado para dados de fabricação e identificação da tag.

Setor trailer Último bloco de cada setor, reservado para armazenar chaves e bits de acesso.

Key A Chave de autenticação armazenada no setor trailer, usada conforme a política de acesso configurada.

Key B Segunda chave de autenticação possível no setor trailer, também dependente das condições de acesso do setor.

Bits de acesso Conjunto de bits e seus complementos que definem permissões de leitura, escrita e operações especiais em blocos e chaves.

Bloco de valor Formato especial de bloco usado para contadores, créditos e débitos, com redundância interna para detecção de erro.

MFRC522 Circuito integrado leitor de RFID da NXP amplamente usado em kits didáticos e módulos de laboratório para ISO/IEC 14443 A.

SPI Sigla de *Serial Peripheral Interface*, barramento serial comum na ligação entre microcontroladores e o leitor MFRC522.

SCK Sigla de *serial clock*, linha de clock do barramento SPI.

MOSI Sigla de *master out, slave in*, linha do barramento SPI usada para dados do controlador para o periférico.

MISO Sigla de *master in, slave out*, linha do barramento SPI usada para dados do periférico para o controlador.

SDA Rótulo usado em muitos módulos RC522 para a linha física de seleção do periférico, embora não represente o papel tradicional de dados seriais em barramentos I2C.

NSS Sigla de *slave select*, sinal de seleção do periférico no barramento SPI.

RST Abreviação de *reset*, sinal usado para reinicialização do leitor.

FIFO Sigla de *First In, First Out*, fila interna usada pelo MFRC522 para transmitir e receber bytes.

CRC Sigla de *Cyclic Redundancy Check*, mecanismo de verificação de integridade usado nos quadros de comunicação.

IRQ Sigla de *Interrupt Request*, sinalização de eventos internos do leitor para o microcontrolador hospedeiro.

MAC Sigla de código de autenticação de mensagem, valor criptográfico usado para verificar integridade e autenticidade de dados.

HMAC Sigla de *Hash-based Message Authentication Code*, forma de MAC construída a partir de função hash e chave secreta compartilhada.

CRYPTO1 Cifra proprietária usada pela MIFARE Classic para autenticação e proteção das transações no nível do cartão.

AES Sigla de *Advanced Encryption Standard*, família de algoritmos simétricos moderna adotada por cartões mais recentes e por muitas camadas de aplicação.

Anticolisão Procedimento que permite ao leitor identificar e selecionar uma tag específica quando várias estão no mesmo campo.

Autenticação mútua Processo em que leitor e tag validam um ao outro antes de liberar acesso a blocos protegidos.

EEPROM Sigla de *Electrically Erasable Programmable Read-Only Memory*, tipo de memória não volátil usada para guardar o conteúdo interno da tag.

Glossário do Capítulo 3

AES — Sigla de *Advanced Encryption Standard*, algoritmo simétrico padronizado usado para cifragem de dados em repouso e em trânsito.

AES-GCM — Modo de operação *Galois/Counter Mode* do AES, que combina cifragem e autenticação em uma única construção (cifra autenticada).

Assinatura digital — Evidência criptográfica produzida com chave privada e verificável com a chave pública correspondente, fornecendo autenticidade e não repúdio.

AuthTag — Etiqueta de autenticação produzida pelo modo GCM, usada para verificar integridade e autenticidade do texto cifrado e dos dados associados.

Autenticidade — Propriedade que fornece evidências de quem gerou ou aprovou uma mensagem ou operação.

Chave privada — Chave secreta de um par assimétrico, usada para assinar ou decifrar conforme o protocolo adotado.

Chave pública — Chave distribuível de um par assimétrico, usada para verificação de assinaturas ou cifragem em cenários específicos.

Cifra autenticada — Construção criptográfica que oferece confidencialidade e verificação de integridade/autenticidade em uma única operação, como AES-GCM.

Colisão — Situação em que duas entradas distintas produzem o mesmo hash, comprometendo a resistência da função hash.

Confidencialidade Propriedade que restringe a leitura do conteúdo apenas a entidades autorizadas.

Contador monotônico Valor que apenas cresce ao longo do tempo de uso de um protocolo, útil para evitar reutilização de estados antigos e mitigar replay.

Criptografia assimétrica Modelo criptográfico baseado em pares de chaves pública e privada, em que operações diferentes usam chaves diferentes.

Criptografia simétrica Modelo criptográfico em que a mesma chave, ou chaves diretamente relacionadas, é usada para cifrar e decifrar.

Curva elíptica Estrutura algébrica usada em criptografia assimétrica, oferecendo boa segurança com chaves menores que alternativas clássicas como RSA.

Derivação de chaves Processo de gerar chaves específicas a partir de uma chave-mestra ou material inicial, usado para compartimentar responsabilidades e limitar impacto de comprometimento.

Digest Resultado de tamanho fixo produzido por uma função hash a partir de uma entrada de tamanho arbitrário; também chamado de resumo.

ECDSA Sigla de *Elliptic Curve Digital Signature Algorithm*, algoritmo de assinatura digital baseado em curvas elípticas.

FIPS Sigla de *Federal Information Processing Standards*, conjunto de padrões publicados pelo NIST para uso em sistemas de informação governamentais e comerciais.

Função hash criptográfica Função unidirecional que transforma dados de tamanho arbitrário em um resumo de tamanho fixo, com propriedades de resistência a pré-imagem e colisão.

GCM Sigla de *Galois/Counter Mode*, modo de operação de cifra de bloco que provê cifragem e autenticação simultâneas.

GMAC Sigla de *Galois Message Authentication Code*, variante do GCM usada apenas para autenticação, sem cifragem.

HMAC Sigla de *Hash-based Message Authentication Code*, construção que combina uma função hash com uma chave secreta para verificar integridade e autenticidade de mensagens.

Integridade Propriedade que permite detectar alteração não autorizada em dados ou mensagens.

IV Sigla de *Initialization Vector*, valor usado para inicializar uma cifra e garantir que a mesma mensagem cifrada com a mesma chave produza resultados distintos.

MAC Sigla de código de autenticação de mensagem, valor criptográfico calculado sobre dados com chave secreta para verificar integridade e autenticidade.

Não repúdio Propriedade que dificulta que o autor de uma assinatura válida negue ter produzido determinada evidência.

NIST Sigla de *National Institute of Standards and Technology*, agência dos EUA responsável por padrões de criptografia e segurança da informação.

Nonce Valor usado uma única vez em um protocolo, normalmente para distinguir execuções e mitigar ataques de replay.

P-256 Curva elíptica padronizada pelo NIST, também chamada de *prime256v1* ou *secp256r1*, amplamente usada em assinaturas ECDSA e troca de chaves.

Pré-imagem Entrada original de uma função hash; resistência à pré-imagem significa que não é viável encontrar a entrada a partir do digest.

Replay Reutilização indevida de uma mensagem antiga válida para tentar obter vantagem ou acesso não autorizado.

RFC Sigla de *Request for Comments*, série de documentos técni-

cos usados para padronização de protocolos e tecnologias da internet.

SHA-256 Função hash da família *Secure Hash Algorithm* que produz um digest de 256 bits (32 bytes), amplamente usada em integridade, HMAC e assinaturas.

Glossário do Capítulo 4

ABI Sigla de *Application Binary Interface*, interface binária que define como interagir com um contrato inteligente a partir de código externo.

Blockchain Estrutura de livro-razão distribuído em que blocos de transações são encadeados por referências criptográficas e validados segundo regras de consenso.

Bloco Agrupamento ordenado de transações e metadados em uma blockchain, vinculado ao bloco anterior por hash criptográfico.

Bytecode Código compilado de um contrato inteligente, executado pela EVM em cada nó validador da rede.

Consenso Conjunto de regras e mecanismos pelos quais uma rede distribuída concorda sobre a ordem e a validade das alterações de estado.

Conta de contrato Conta controlada pelo código de um contrato inteligente implantado na blockchain, sem chave privada externa.

Conta externa Conta controlada por uma chave privada de usuário ou serviço, capaz de iniciar transações na rede.

Contrato inteligente Programa armazenado na blockchain e executado de forma determinística pela plataforma, sem possibilidade de alteração após o deploy.

Deploy Processo de implantar um contrato inteligente na blockchain, criando uma nova conta de contrato com o bytecode compilado.

Estado on-chain Conjunto de dados atualmente mantido e validado pela blockchain como estado oficial do protocolo.

Ethereum Plataforma blockchain programável que permite a execução de contratos inteligentes em um ambiente de máquina virtual distribuída.

Evento Registro emitido por um contrato inteligente durante a execução, armazenado nos logs da transação e indexável por aplicações externas.

EVM Sigla de *Ethereum Virtual Machine*, máquina abstrata que executa o bytecode dos contratos de forma determinística em todos os nós da rede.

Função de escrita Função de contrato que altera estado on-chain, gerando uma transação que consome gas e precisa ser minerada.

Função de leitura Função de contrato marcada como `view` ou `pure`, que consulta estado sem alterá-lo e sem consumir gas.

Gas Unidade abstrata de custo computacional usada para medir e cobrar execução e armazenamento em plataformas como Ethereum.

Hardhat Framework de desenvolvimento para contratos Solidity, que inclui compilação, testes, deploy e rede local para simulação.

Hash do bloco Referência criptográfica que vincula cada bloco ao anterior, formando a cadeia que dá nome à blockchain.

Imutabilidade Propriedade de dados ou código on-chain que não podem ser alterados após a gravação, garantindo integridade histórica.

Mapping Estrutura de dados do Solidity equivalente a uma tabela hash, usada para associar chaves a valores com acesso O(1).

Mineração Processo pelo qual nós da rede validam transações e blocos,

aplicando regras de consenso para determinar o próximo estado.

Modifier Construção do Solidity que adiciona verificações pré ou pós-execução a funções de contrato, como controle de permissão.

Nó Instância participante de uma rede blockchain que valida, propaga, consulta ou armazena o estado compartilhado.

Off-chain Dados ou computações que ocorrem fora da blockchain, tipicamente em serviços auxiliares, para reduzir custo e preservar privacidade.

On-chain Dados ou computações que ocorrem dentro da blockchain, com garantias de verificabilidade, imutabilidade e rastreabilidade.

Receipt Recibo de uma transação confirmada, contendo hash, status, gas consumido e logs de eventos emitidos pelo contrato.

Relayer Componente intermediário que conecta dispositivos de borda (como o ESP32) à blockchain, encaminhando transações e consultas.

Solidity Linguagem de programação orientada a contratos, usada para escrever contratos inteligentes na plataforma Ethereum e compatíveis.

Struct Tipo de dado composto em Solidity que agrupa campos relacionados em uma única estrutura, como o `AccessRecord` do laboratório.

Transação Operação submetida à blockchain para transferir valor, invocar código ou alterar estado, assinada pela conta remetente.

Glossário do Capítulo 5

CMake Sistema de configuração e geração de build usado pelo ESP-IDF para descrever componentes, dependências e alvos de compilação.

CTR_DRBG Mecanismo determinístico de geração de números pseudoaleatórios baseado em cifra de bloco, usado para apoiar geração de chaves e assinaturas.

ESP-IDF Sigla de *Espressif IoT Development Framework*, framework oficial de desenvolvimento para dispositivos da família ESP32.

ESP32 Família de sistemas em chip da Espressif voltada a aplicações embarcadas conectadas, com suporte a Wi-Fi, periféricos seriais e execução local de firmware.

Event Loop Mecanismo central do ESP-IDF para entrega de eventos do sistema, como mudança de estado da rede ou desconexão do Wi-Fi.

FreeRTOS Sistema operacional de tempo real usado como base de agendamento, tarefas, filas e sincronização no ESP-IDF.

Firmware Camada de software executada diretamente no dispositivo embarcado, responsável por controlar periféricos, rede e lógica local.

GPIO Sigla de *General-Purpose Input/Output*, pinos programáveis usados para entrada e saída digital.

Mbed TLS Biblioteca criptográfica integrada ao ESP-IDF, usada para hash, TLS, geração de chaves e assinaturas digitais.

NVS Sigla de *Non-Volatile Storage*, mecanismo de armazenamento persistente usado pelo ESP-IDF para dados locais e apoio a componentes como Wi-Fi.

PlatformIO Ecossistema de build e automação que integra toolchain, gerenciamento de plataforma, upload e monitor serial para projetos embarcados.

SDK Sigla de *Software Development Kit*, conjunto de bibliotecas, ferramentas e documentação necessário para desenvolver sobre uma plataforma.

SoC Sigla de *System on Chip*, circuito integrado que reúne CPU, memória e periféricos em um único chip.

SPI Host Controlador SPI do lado mestre no ESP32, responsável por iniciar e coordenar transações no barramento.

Station Mode Modo em que o ESP32 atua como cliente Wi-Fi e se conecta a um ponto de acesso já existente.

Task Unidade de execução agendada pelo FreeRTOS, comparável a uma thread em sistemas embarcados.

Referências Bibliográficas

Tim Berners-Lee, Roy Fielding, and Larry Masinter. Rfc 3986: Uniform resource identifier (uri): Generic syntax. IETF RFC, 2005. URL https://www.rfc-editor.org/rfc/rfc3986.

Alija Bobija. esp-idf-rc522. GitHub repository and ESP-IDF component, 2026. URL https://github.com/abobija/esp-idf-rc522. Acesso em: 2026-03-07.

Vitalik Buterin. A next-generation smart contract and decentralized application platform. White paper, 2014. URL https://ethereum.org/en/whitepaper/. Acesso em: 2026-03-06.

Lily Chen, Dustin Moody, Andrew Regenscheid, Angela Robinson, and Karen Randall. Nist sp 800-186: Recommendations for discrete logarithm-based cryptography: Elliptic curve domain parameters. NIST Special Publication, 2023. URL https://doi.org/10.6028/NIST.SP.800-186. Acesso em: 2026-03-06.

Gerhard de Koning Gans, Jaap-Henk Hoepman, Flavio D. Garcia, and Erik Verheul. A practical attack on the mifare classic. In *Smart Card Research and Advanced Applications*. Springer, 2008. URL https://arxiv.org/abs/0803.2285. Acesso em: 2026-03-06.

Morris Dworkin. Nist sp 800-38d: Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. NIST Special Publication, 2007. URL https://doi.org/10.6028/NIST.SP.800-38D. Acesso em: 2026-03-06.

Espressif Systems. Mbed tls - esp32 - esp-idf programming guide. Official documentation, 2026a. URL https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mbedtls.html. Acesso em: 2026-03-07.

Espressif Systems. Non-volatile storage library - esp32 - esp-idf programming guide. Official documentation, 2026b. URL https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/storage/nvs_flash.html. Acesso em: 2026-03-07.

Espressif Systems. Esp-idf programming guide - esp32. Official documentation, 2026c. URL https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html. Acesso em: 2026-03-07.

Espressif Systems. Spi master driver - esp32 - esp-idf programming guide. Official documentation, 2026d. URL https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/spi_master.html. Acesso em: 2026-03-07.

Espressif Systems. Wi-fi - esp32 - esp-idf programming guide. Official documentation, 2026e. URL https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_wifi.html. Acesso em: 2026-03-07.

Espressif Systems. Esp32 wi-fi & bluetooth soc. Official product page, 2026f. URL https://www.espressif.com/en/products/socs/esp32. Acesso em: 2026-03-07.

Ethereum Foundation. Evm opcodes. Ethereum.org documentation, 2026. URL https://ethereum.org/en/developers/docs/evm/opcodes/. Acesso em: 2026-03-06.

Klaus Finkenzeller. *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication*. Wiley, 3 edition, 2010.

Flavio D. Garcia, Gerhard de Koning Gans, Ruben Muijrers, Peter van Rossum, Roel Verdult, Ronny Wichers Schreur, and Bart Jacobs. Dismantling mifare classic. In *Computer Security – ESORICS 2008*. Springer, 2008. URL https://flaviodgarcia.com/publications/Dismantling.Mifare.pdf. Acesso em: 2026-03-06.

Hugo Krawczyk, Mihir Bellare, and Ran Canetti. Rfc 2104: Hmac: Keyed-hashing for message authentication. IETF RFC, 1997. URL https://www.rfc-editor.org/rfc/rfc2104.

Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. White paper, 2008. URL https://bitcoin.org/bitcoin.pdf. Acesso em: 2026-03-06.

National Institute of Standards and Technology (NIST). Fips 180-4: Secure hash standard (shs). Federal Information Processing Standards Publication, 2015. URL https://doi.org/10.6028/NIST.FIPS.180-4. Acesso em: 2026-03-06.

National Institute of Standards and Technology (NIST). Fips 186-5: Digital signature standard (dss). Federal Information Processing Standards Publication, 2023a. URL https://doi.org/10.6028/NIST.FIPS.186-5. Acesso em: 2026-03-06.

National Institute of Standards and Technology (NIST). Fips 197: Advanced encryption standard (aes). Federal Information Processing Standards Publication, 2023b. URL https://doi.org/10.6028/NIST.FIPS.197-upd1. Acesso em: 2026-03-06.

NXP Semiconductors. Mf1p(h)x2: Mifare plus ev2. Product data sheet, Rev. 3.1, 2021. URL https://www.nxp.com/docs/en/data-sheet/MF1P%28H%29x2.pdf. Acesso em: 2026-03-06.

NXP Semiconductors N.V. Mfrc522: Standard performance mifare and ntag frontend. Product data sheet, Rev. 3.9, 2016. URL https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf. Acesso em: 2026-03-06.

NXP Semiconductors N.V. Mf1s50yyx_v1: Mifare classic ev1 1k - mainstream contactless smart card ic for fast and easy solution development. Product data sheet, Rev. 3.2, 2018. URL https://www.nxp.com/docs/en/data-sheet/MF1S50YYX_V1.pdf. Acesso em: 2026-03-06.

PlatformIO. Espressif iot development framework. PlatformIO documentation, 2026a. URL https://docs.platformio.org/en/latest/frameworks/espidf.html. Acesso em: 2026-03-07.

PlatformIO. Espressif 32. PlatformIO documentation, 2026b. URL https://docs.platformio.org/en/latest/platforms/espressif32.html. Acesso em: 2026-03-07.

Thomas Pornin. Rfc 6979: Deterministic usage of the digital signature algorithm (dsa) and elliptic curve digital signature algorithm (ecdsa). IETF RFC, 2013. URL https://www.rfc-editor.org/rfc/rfc6979.

Solidity Team. Solidity documentation. Official documentation, 2026. URL https://docs.soliditylang.org/. Acesso em: 2026-03-06.

Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Yellow Paper, 2025. URL https://ethereum.github.io/yellowpaper/paper.pdf. Acesso em: 2026-03-06.

World Wide Web Consortium (W3C). Decentralized identifiers (dids) v1.0: Core architecture, data model, and representations. W3C Recommendation, 2022. URL https://www.w3.org/TR/did-core/. Acesso em: 2026-03-06.

World Wide Web Consortium (W3C). Verifiable credentials data model v2.0. W3C Recommendation, 2023. URL https://www.w3.org/TR/vc-data-model-2.0/. Acesso em: 2026-03-06.

World Wide Web Consortium (W3C). Decentralized identifier (did) specification registries. W3C Working Group Note, 2024. URL https://www.w3.org/TR/did-spec-registries/. Acesso em: 2026-03-06.

World Wide Web Consortium (W3C). Decentralized identifier resolution (did resolution) v0.3. W3C Working Draft, 2026. URL https://www.w3.org/TR/did-resolution/. Acesso em: 2026-03-06.